TECHNISCHE UNIVERSITÄT
CHEMNITZ

Department of Computer Science
Chair of Distributed and Self-organizing Systems

Master Thesis

Web-based Platform for Automated Web User Interfaces Evaluation

Calvin Liusnando

Chemnitz, 15 April 2024

Examiner: Prof. Dr.-Ing Martin Gaedke

Supervisor: Dr.-Ing Sebastian Heil

Liusnando, Calvin
Web-based Platform for Automated Web User Interfaces Evaluation
Master Thesis, Department of Computer Science
Chemnitz University of Technology, 15 April 2024

Abstract

The evaluation of user interfaces is a crucial aspect for the success of software products, as it enables a structured approach to iteratively build and improve the user experience to achieve the desired goals in terms of usability, complexity, and aesthetics. Traditional evaluation methods demand significant resources in terms of time and/or human expertise. Automated evaluation methods overcome these constraints, and previous works involving various models from both traditional machine learning and recent deep learning approaches have shown promising results in evaluating user interfaces.

To make existing evaluation models more readily available, this thesis aims to create a web-based platform allowing the usage of established metrics & models for automatic evaluation of web user interfaces. The platform architecture must be able to process textual and visual inputs, allow the user to select the metrics that are computed, support concurrent execution of the models, optimize shared preprocessing tasks, visually represent the results as they become available, and should allow to reference to previous results via persistent URLs and to extend the platform with new metrics and models.

The objective of this thesis is the creation of a solution or the combination of existing techniques to solve the problem of facilitating the use of automatic web user interface assessment models through a web-based platform as described above. This comprises the analysis of the state of the art of UI assessment tools as well as the demonstration of the solution by implementation and suitable experimental evaluations focusing on the usability and system performance of the assessment platform.

Table of Contents

List of Figures ix

List of Tables xi

List of Listings xiii

List of Abbreviations xv

1 Introduction 1
1.1 Situation 1
1.2 Motivation 3
1.3 Problem Statement 4
1.4 Objectives and Scope 6
1.5 Outline 7
1.6 Summary 8

2 State of the Art 9
2.1 Requirements Analysis 9
2.1.1 Functional Requirements 10
2.1.2 Non-functional Requirements 14
2.2 Related Works 16
2.2.1 Metric-Based Approaches 17
2.2.1.1 Single-Metric Approaches 18
2.2.1.2 Multi-Metric Approaches 26
2.2.2 UI Evaluation Platform 30
2.3 Analysis of Assessment Results 33
2.3.1 Analysis of Assessment Results by Requirements 33
2.3.2 Limitations of Existing Works 36
2.4 Summary 37

3 Concept 39
3.1 Software Architecture 40
3.1.1 Frontend 42

3.1.2 Backend 46
3.1.3 Database 49
3.1.4 Proxy Server 51
3.2 Activity Diagram 52
3.3 Summary 53

4 Implementation **55**
4.1 Platform and Core Technologies 55
4.2 Metrics Selection and Implementation 56
4.2.1 Metrics Overview 56
4.2.2 Metric Interface 62
4.2.3 Metrics Specification 63
4.3 Software Components Implementation 64
4.3.1 Frontend 64
4.3.2 Backend 66
4.4 Metrics Extension 67
4.5 Summary 69

5 Evaluation **71**
5.1 Experimental Evaluations 71
5.1.1 Usability Evaluation 71
5.1.2 System Performance Evaluation 78
5.2 Requirements Evaluation 81
5.3 Summary 84

6 Conclusion and Outlook **85**
6.1 Thesis Summary 85
6.2 Contributions 86
6.3 Critical Consideration and Future Work 86
6.3.1 Future Work 88

Bibliography **90**

Appendix A Implementation Details **103**

Appendix B Evaluation Materials **105**
B.1 Initial Questionnaire 105
B.2 Task List for Usability Evaluation 107

B.3 Initial Questionnaire Responses 109
B.4 Task Completion Times 110
B.5 SUS Evaluation Data..... 110
B.6 UEQ Evaluation Data 111
B.7 AttrakDiff Evaluation Data 113
B.8 Computation Times of Feature Congestion and Subband Entropy 114

List of Figures

Figure 3.1: Component Diagram of the Software Architecture Overview 41
Figure 3.2: Database Schema for Persisting WUI Evaluation Results 50
Figure 4.1: Display of metric evaluation results 65
Figure 4.2: Metric Extension Interface of WUI Auto Evaluate 68
Figure 5.1: Aggregated UEQ Evaluation Data 75
Figure 5.2: Mean values of the word pairs of AttrakDiff questionnaire 76
Figure 5.3: Metric computation times (excluding m10 and m11) 79
Figure B.1: Self-assessment section of the initial questionnaire 106
Figure B.8.1: Computation times of the feature congestion metric..... 114
Figure B.8.2: Computation times of the subband entropy metric..... 114

List of Tables

Table 2.1: List of requirements 10
Table 2.2: Overview of assessed metric-based approaches with regard to the nature of output and evaluation methodology 18
Table 2.3: Evaluation of QUESTIM 27
Table 2.4: Evaluation of WAVE 28
Table 2.5: Evaluation of Kobold 30
Table 2.6: Evaluation of WUI Measurement Platform 31
Table 2.7: Evaluation of AIM 33
Table 2.8: Assessment results overview 33
Table 4.1: ID, Name and References of the Implemented Metrics 58
Table 5.1: Mean and standard deviation of task completion times 74
Table 5.2: Statistical overview of UEQ scales 75
Table 5.3: Mean and stdev deviation of metrics computation time 79
Table 5.4: Mean and stdev of metrics computation time for Scenario 1 80
Table 5.5: Mean and stdev of metrics computation time for Scenario 2 81
Table 5.6: WUI Auto Evaluate in comparison to state of the art 82
Table B.3.1: Questionnaire responses excluding Q4 109
Table B.3.2: Questionnaire responses for Q4 109
Table B.4: Task completion times 110
Table B.5: SUS evaluation data 110
Table B.6.1: UEQ raw evaluation data 111
Table B.6.2: UEQ results for each item of the questionnaire 112
Table B.7: AttrakDiff raw evaluation data 113

List of Listings

Listing 3.1.2: Example of the DOM Analyzer output 48
Listing 4.1: Metric Interface 62
Listing A.1: Dynamic Import for Metric Implementations 103
Listing A.2: Example of a valid metric configuration 104

List of Abbreviations

AIM	Aalto Interface Metrics
ANN	Artificial Neural Network
API	Application Programming Interface
CNN	Convolutional Neural Network
DOM	Document Object Model
GUI	Graphical User Interface
HCI	Human-Computer Interaction
HTML	Hypertext Markup Language
IP	Internet Protocol
JSON	JavaScript Object Notation
NIMA	Neural IMage Assessment
OCR	Optical Character Recognition
PNG	Portable Network Graphics
SUS	System Usability Scale
UE	Usability Evaluation
UEQ	Usability Evaluation Questionnaire
UI	User Interface
UIED	User Interface Element Detection
UMSI	Unified Model of Salience and Importance

URL	Uniform Resource Locator
UUID	Universally Unique Identifier
WCAG	Web Content Accessibility Guidelines
WUI	Web User Interface
WWW	World Wide Web

1 Introduction

The topic of this thesis is the creation of a web-based platform to facilitate the use of established metrics and assessment models to automate the evaluation of web-based graphical user interfaces (GUI), commonly referred to as web user interfaces (WUI). This chapter starts with a brief overview of the historical development and current state of the related study field, providing context and motivation for the need for automated WUI evaluation in today's digital landscape. The problem statement and the research problems to be addressed within this thesis are detailed within this chapter, serving as the framework that guides the research and work to be undertaken within this thesis.

1.1 Situation

As the gateway through which users interact with the content, the user interface (UI) stands as a crucial element in determining the effectiveness and overall user experience of these systems. Previous studies in the field of Human-computer interaction (HCI) have highlighted the importance of a well-executed user interface. For instance, one study found a 2:1 savings-to-cost ratio for a relatively small development project and a 100:1 savings-to-cost ratio for a large development project [1]. Another study estimated that the benefits of usability engineering are between 1000 to 5000 times greater than the associated costs [2]. Conversely, poor user interface has been identified as a problem as it leads to increased user frustration and dissatisfaction, along with loss of productivity and efficiency in business [3].

Consequently, the evaluation of user interfaces is of great importance and attracts high interest, leading to the development of a wide range of usability evaluation (UE) methods over the last few decades. UE methods, in a broader sense, involved several common key activities [10, 11]:

- *Capture*, referring to the process of collecting usability data, such as "task completion time, errors, guideline violation, and subjective ratings" [10]

- *Analysis*, which entails examining the collected data to identify the usability problems of the given input.
- *Critique*, focusing on evaluating the findings from the analysis, and suggesting solutions or improvements to mitigate the found issues.

Prominent examples of these methods include heuristic evaluation, which involves evaluators reviewing interfaces based on predefined usability principles [4], think-aloud protocol, where users are asked to verbalize their thought process while interacting with the interface [5], and cognitive walkthroughs, which focuses on evaluating the ease of performing tasks in a system from the perspective of a new user [6].

These aforementioned techniques, while indeed can provide valuable insight into the usability of the targeted interface, share common limitations. They typically require considerable resources in terms of time and human expertise. One study suggests that the sample size (i.e. test users for think-aloud protocol and evaluators for heuristic evaluation and cognitive walkthrough) of 10±2 is needed to reach 80% of the overall discovery rate [7]. This finding indicates that a substantial investment in human expertise is necessary to achieve a comprehensive result, rendering these methods impractical for projects constrained by limited resources.

Furthermore, several studies [8,9] have demonstrated that different evaluators, even when using the same methods, often identify substantially different sets of usability issues in the system. This variability suggests that there exists a level of inconsistency and unpredictability within the evaluation process, which could potentially undermine the reliability of the results.

Automating the evaluation process has been identified as a viable alternative solution to overcome the constraints inherent in the non-automated evaluation methods. Ivory and Hearst [10] highlighted several potential advantages of this approach. These include reducing the cost of usability evaluation, the need for evaluation expertise among individual evaluators, increasing consistency of the uncovered errors, and enabling the incorporation of the evaluation within the design phase of the UI development [10].

Given its potential benefits, there has been a growing research interest and desire to automate the evaluation of UI in a systematic and comprehensive manner. Numerous methodologies and frameworks with diverse approaches have been developed over the last few years. As noted by Chi et al. [12], these approaches generally can be grouped into two different categories. The first category focuses on ensuring that the interface

adheres to the best practices, standards, and guidelines such as the Web Content Accessibility Guidelines (WCAG)[1]. Examples include automated accessibility checker tools such as WAVE[2] and Axe accessibility testing tools[3].

The other category aims to compute certain aspects of the interface through assessment models and algorithms based on the provided interface design. One notable example is the *Aalto Interface Metrics Service*[4] (AIM) [14]. This service computationally evaluates the input GUI based on empirically validated web metrics and models, which are pooled from various research studies. These metrics are separated into four categories: accessibility, aesthetics, color perception, and perceptual fluency [14]. The result given is an overview of what the chosen metric does and an indicator of empirical evidence for its predictive power [14].

Furthermore, rapid advancement in artificial intelligence, machine learning, and deep learning has opened up new avenues for WUI evaluation. Recent research within these domains has demonstrated the ability of these approaches to quantify certain aspects of WUI, such as visual complexities [15, 17, 43, 50], aesthetics [16, 17, 19, 42, 50], and accessibility [18]. These developments represent significant advancement and exciting opportunities in the field of WUI evaluation, as they showcase the potential of enabling a more accurate and comprehensive evaluation of WUI.

1.2 Motivation

There have been a lot of advancements and research findings on the topic of automating WUI evaluation. However, apart from accessibility checker tools, most of this progress remains within the confines of academic research and comprises experimentally validated implementation. Furthermore, most of these implementations are scattered across various disciplines, and not readily accessible. This situation underscores an imperative need to translate these research findings (i.e., web metrics and WUI assessment models), into a platform that is accessible and practical for everyday use in UI design and evaluation scenarios. Consolidating these advancements into one common platform will help in simplifying the process for both researchers and practitioners to make use of and

[1] https://www.w3.org/WAI/standards-guidelines/wcag/ [Retrieved: Nov 22, 2023]
[2] Available at: https://wave.webaim.org/ [Retrieved: Dec 16, 2023]
[3] Available at: https://www.deque.com/axe/ [Retrieved: Dec 16, 2023]
[4] Available at: https://interfacemetrics.aalto.fi/ [Retrieved: Dec 10, 2023]

further explore these metrics and models, and potentially perform comprehensive UI evaluation with minimal effort and resources.

The example mentioned in the previous section, AIM, stands out as the most prominent web-based platform, to the author's knowledge, that is readily accessible for evaluation of GUI based on various metrics and UI assessment models. However, while the platform does support the computation of numerous metrics, there are some limitations, as noted by the author:

1. It only accepts one input at a time. Therefore, it can only compute the metrics of a single input per session, making using the platform somewhat tedious when there is a large number of screenshots to be processed.
2. The evaluation results are not persistently saved. Once a user closes the website, these results are lost, necessitating re-computation.
3. The computation of selected metrics is performed sequentially, which leads to relatively long processing times when multiple metrics need to be calculated or when complex computations are required.
4. While the platform itself is designed with extensibility in mind, it is currently not possible to add new metrics or models within the platform, and direct changes in the source code are necessary.

Considering these limitations, developing a new web-based platform that addresses and/or alleviates these constraints presents significant research potential and challenges, which will be investigated in the subsequent sections and chapters of this thesis.

1.3 Problem Statement

The central research problem that this thesis aims to solve is the lack of and limitations of existing tools and platforms in facilitating the use of existing research findings such as web metrics and WUI assessment models for automation of UI evaluation. To tackle the overarching central problem, the following sub-problems are identified and will be investigated within the scope of this thesis:

- Suitable approaches to improve the performance of metric computations.
- The implementation of an appropriate solution that facilitates the extension of new metrics within the developed platform.

The development of such a platform comes with a set of challenges which must be addressed to ensure the successful development of the platform. Namely, these challenges are:

- **The creation an UI which is intuitive, aesthetically pleasing, and accessible to a diverse set of users.**

  This poses a significant challenge as it requires deep understanding and investigation of insights from prior studies in the fields of HCI and user experience designs, finding the right balance between the visual aesthetic and functional usability of the platform, and adhering to guidelines outlined in accessibility frameworks such as WCAG for creating an accessible web content.

- **Implementation of a versatile system which can handle diverse input types and multiple inputs at once.**

  This challenge is a quintessential problem in system architecture design: it necessitates the development of a flexible and comprehensive system capable to handle and multiple inputs simultaneously. Furthermore, the system must also be able to verify the validity of each input based on predefined criteria.

- **Achieving better performance in the WUI metrics computation.**

  Since the WUI metrics computation might involve machine learning / deep learning model(s) which can be resource expensive, computing multiple metrics can be potentially time consuming and adversely affect the usability of the platform. Therefore, it is crucial to investigate techniques which can optimize the processing workflows and speed up the computations.

- **Design and implementation of a system architecture that can facilitate the extension of new metrics and models.**

  Creating an extensible system is often a complex task as it demands a careful consideration of various key factors. [24, 25] highlighted challenges that come with extensibility:

  - The addition of extensibility often increases the complexity of the software, making it more difficult to develop, test, and maintain.
  - Extensible designs and implementations are more time consuming, and therefore come with higher associated costs.

  - Knowledge on the way the system is going to be extended in the future is crucial to successfully design and implement an extensible system, and this is often not easy to anticipate.
  - Extensibility often decreases the system performance, necessitating thoughtful planning and fine-tuning to achieve the optimal balance between the system's extensibility and its performance.

- **Achieving persistent storage and efficient retrieval of the evaluation results.**

  As persistent storage necessitates a robust database solution, the challenges arise from designing a database architecture that can scale well with growing volume of data while simultaneously maintaining data integrity and security. The added complexity arises from the potential unstructured nature of the data, which comes with numerous information retrieval challenges, and addressing them requires deep understanding and appropriate implementation of data management principles and solutions that have been developed from previous studies and works [26].

1.4 Objectives and Scope

The main objective of this thesis is to solve the problem of facilitating the easier use of WUI metrics and assessment models for automated UI evaluation through a web-based platform by addressing the challenges as described in section 1.3. Furthermore, to assess the quality and usefulness of the developed solution, a suitable experimental evaluation is to be conducted, focusing on the usability aspects and system performance of the developed platform.

The scope of this thesis is derived and defined from the research problems posed in section 1.3. However, these following topics, which are closely or somewhat related to the topic of this thesis, is considered as out of scope:

- This thesis only considers graphical user interface, and the evaluation of any other type of user interfaces (such as command-line interface, natural language interface, etc.) will not be addressed, as they have different design principles and require evaluation metrics which will not be discussed in this thesis.
- This thesis will focus on WUIs. As noted by Ivory & Hearst [10] and Charfi et al. [23], WUIs have some distinct characteristics which makes it different compared

to non-web-based UIs, such as WIMP (windows, icons, pointer, and mouse) interfaces or mobile device-based UI. While theoretically the developed platform can accept graphical UI not designed for the web, the evaluation results might not be as accurate since the metrics and models to be used by the platform will be intended for the assessment of WUI.

- This thesis does not aim to define a new metric, create a new assessment model, or improve upon existing assessment models. Instead, it focuses on making use of previous findings from various research fields and to integrate them into a single unifying web-based platform.
- Any kind of aggregation of evaluation results to form a holistic assessment will not be pursued. This thesis only focuses on the development of a platform capable of evaluating WUI based on the selected metrics, without delving into the complexities of combining disparate evaluation results into a singular score or result.

1.5 Outline

The remainder of the thesis is structured as follows: In chapter 2, requirements are elicited based on the motivation, research problems, and challenges outlined in this chapter. It also reviews the state of the art in the field of automated WUI evaluation with regard to the requirements, and shortcomings of the existing approaches are identified. Chapter 3 details the concept of the proposed solution to address the shortcomings. Chapter 4 present the technical details and approaches taken to realize the conceptual framework. The proposed solution is then evaluated as described in chapter 5. Finally, Chapter 6 summarizes the results, insights, and contributions of the thesis, addressing limitations and providing suggestions for future research.

1.6 Summary

This chapter has provided an overview of user interface evaluation, highlighting its importance and the challenges involved. It has been recognized that automating the evaluation process is a promising approach to address the drawbacks of traditional, non-automated methods. Despite significant advancements and research in this field, there remains a notable gap: the absence of a practical and user-friendly platform that can efficiently utilize the existing web metrics and assessment models for WUI evaluation. This thesis aims to bridge this gap by the creation of a web-based platform to solve the research problems outlined in the section 1.3. The scope of this thesis is then defined based on this objective. The outline section describes the structure of the thesis and a short overview on the content for each of the following chapters.

2 State of the Art

This chapter further investigates the research problems and challenges raised in the introduction. A set of requirements as well as its evaluation scheme is formulated to tackle the central research problem, along with the sub-problems and the associated challenges. Existing relevant work are then investigated and evaluated based on the defined requirements and evaluation scheme.

2.1 Requirements Analysis

In this section, requirements are elicited based on the motivation, research problems, and challenges outlined in the introduction chapter. These requirements will be used to measure what the state-of-the-art approaches have achieved and are currently lacking. These requirements are categorized into two groups:

- **Functional requirements**, which specify the requirements encompassing the functionalities that the solution must support to meet its intended purpose.
- **Non-functional requirements,** which specify the requirements pertaining to the qualities and attributes of the solution.

Each of the requirement is to be mapped onto a three-level evaluation scheme, namely: not satisfied, partially satisfied, and satisfied. In addition, each requirement has its own requirement level in accordance with RFC 2119 levels of MUST and SHOULD [29]. MUST means the requirement is absolute and fulfillment is mandatory, while SHOULD means the fulfillment of the requirement is recommended. The table 2.1 provides an overview of the requirements which will be detailed in the following paragraphs.

ID	Requirement Description	Level
Functional Requirements		
F1	Diverse WUI Input Types Support	MUST
F2	Selectable Metrics	MUST
F3	Efficient Input Pre-processing	MUST
F4	Parallel Processing of Metrics Computation	MUST
F5	Parallel Processing of Evaluation Requests	SHOULD
F6	Persistent Evaluation Results	SHOULD
Non-Functional Requirements		
NF1	Extension of New Metrics/Models	MUST
NF2	Accessible UI Design	SHOULD

Table 2.1: List of requirements

2.1.1 Functional Requirements

The following five requirements detail the concrete functionalities that an ideal solution need to possess.

F1 – Diverse WUI Input Types Support

The solution needs to support various input options. For the WUI input, it must support the input in the form of a uniform resource locator (URL) to enable users to analyze web pages by simply providing their URL. It also needs to accommodate other formats, such as Hypertext Markup Languages (HTML), Portable Network Graphics (PNG), Joint Photographic Experts Group (JPEG) files to cater to different user needs and scenarios. This versatility of input options is critical, as specific metrics may require particular types of inputs.

For illustration, the computation of certain metrics [15, 30, 31, 34] requires source code analysis, necessitating the availability of extractable HTML from the given input. Other metrics, such as the measures of visual clutter and color variability described by Miniukovich and De Angeli [34], rely on the file size of images in the JPEG format (for visual clutter) and in the PNG format (for color variability). In [43], JPEG and PNG file sizes are used as variables in a regression model to assess subjective evaluations of complexity, aesthetics, and orderliness of WUIs. These examples highlight the necessity for the solution to support multiple file types, as it is required to enable the computation of a wide range of metrics from various approaches with different input requirements.

The requirement is satisfied if the solution supports URL and at minimum one other different file format (such as HTML, JPEG, PNG) as the WUI input. The requirement is partially satisfied if the solution only supports URL as the WUI input. Finally, the requirement is considered not satisfied if the solution does not support URL as an input format. URL support is regarded as the highest priority because other file formats can be extracted from a URL.

F1 is classified at the MUST level as it is crucial to ensure that the solution can accommodate the computation of a diverse array of metrics, which might come with differing input prerequisites.

F2 - Selectable Metrics

The solution must have at minimum more than one metrics for evaluation. Furthermore, the solution needs to allow users to select and deselect which metrics to be evaluated depending on their needs, allowing for a more targeted and efficient evaluation.

The requirement is satisfied if the solution offers multiple metrics for evaluation, all of which are selectable and deselectable for users. The requirement is partially satisfied if the solution provides multiple metrics for evaluation, but not all metrics are selectable and deselectable. Finally, the requirement is not satisfied if the metrics are predefined and non-selectable, or if the solution only supports evaluation of a single metric.

F2 is classified at the MUST level because the ability to select and deselect which metrics to be evaluated is essential for optimizing both the user experience and performance of the solution. This feature enables users to customize their analysis to their specific needs, ensuring that only relevant, selected metrics are computed. This approach effectively avoids unnecessary computations on non-selected metrics, enhancing the overall efficiency of the solution.

F3 - Efficient Input Pre-processing

Computation of the selected metrics might require pre-processing tasks for the input WUI. In this thesis, pre-processing tasks are defined as initial processing steps that prepare or transform the input WUI into a format or state that is suitable for subsequent metric computation. When required, these pre-processing tasks, for examples: image conversion to grayscale [32, 36], image segmentation [15, 32, 33, 34, 35], UI object recognition [32, 33, 35, 36], and text recognition [32, 33, 35], need to be performed only once per WUI input, with the intermediate results distributed to the subsequent dependent steps in the evaluation process. This requirement aims to optimize the overall performance of the solution by eliminating redundant computations.

The requirement is satisfied if the solution performs each of the necessary pre-processing tasks only once for each WUI input, with the intermediate results effectively used in the subsequent steps that depend on them. For illustration, consider the scenario where computation of metric m requires pre-processing tasks a, b, and c. In this case, the requirement is satisfied when each of these tasks, a, b, and c, is performed only once, regardless of the number of subsequent dependent steps in the evaluation results that may require their results. The requirement is also considered satisfied if preprocessing tasks are not needed to evaluate the input. The requirement is partially satisfied if the solution demonstrates a basic level of optimization in handling of the pre-processing tasks, such that some, but not all tasks are only computed once and can be efficiently distributed, while other tasks might require repetitive execution for the same WUI input due to the inefficiencies in the evaluation process. Finally, the requirement is considered not satisfied if the solution requires re-computations of the necessary pre-processing tasks in each of the steps in the evaluation process.

F3 is classified at the MUST level because performance optimization of the metrics computation is part of the research problems that this thesis aims to address. Ensuring efficient input pre-processing directly contributes to this goal as it would enhances the overall computational efficiency of the solution.

F4 - Parallel Processing of Metrics Computation

When multiple metrics are selected for evaluation, the computations for each of the selected metric need to be run independently in parallel. Furthermore, the evaluation result of a metric must be presented to the user immediately once it is available. This requirement aims to enhance the performance and efficiency of the solution as parallel

computation would theoretically speed up the overall runtime of the evaluation process by processing the selected metrics simultaneously.

The requirement is satisfied if the solution fulfill all the criteria mentioned above. The requirement is partially satisfied if the solution demonstrates only limited parallel processing capabilities, where not all the selected metrics can be processed in parallel. It is also considered partially satisfied if the solution can run the computations of the selection metrics in parallel, but the results are not shown immediately when it is available. Finally, the requirement is considered not satisfied if the solution does not support parallel processing of metrics computation.

F4 is classified at the MUST level because the parallel computation of metrics is one of the research problems that this thesis aims to address. The performance improvement which comes with this optimization will enable the solution to handle comprehensive WUI evaluation involving multiple metrics more swiftly, ultimately enhancing the effectiveness and usability of the solution.

F5 - Parallel Processing of Evaluation Requests

In this context, an evaluation request refers to a user-initiated action for evaluating a WUI input through the solution. The solution should be designed to effectively support processing of multiple evaluation requests simultaneously at a time. This means that the solution should have the capability to accept and process multiple WUI inputs in parallel at a time. Furthermore, the processing of each request should be independent, such that errors in the evaluation of one WUI should not block the evaluation process for other WUIs.

The requirement is satisfied if the solution fulfill all the criteria mentioned above. The requirement is partially satisfied if the solution can accept multiple WUI inputs at once and process them in parallel, but errors in the evaluation of one WUI block the evaluation process for other WUIs. Finally, the requirement is considered not satisfied if the solution does not support the submission of multiple input at once or if it can only handle these multiple inputs sequentially.

F5 is classified at the SHOULD level because while it would enhance the efficiency and user experience, it is not essential for the core functionality of the solution. Therefore, while valuable, this requirement is recommended but not mandatory, reflecting its status as desirable but not critical enhancement.

F6 - Persistent Evaluation Results

The solution should support the persistence of the evaluation results. This means that once the metrics of the WUI has been computed, the results are to be stored in a manner that allows for future retrieval. The results then should be accessible through a persistent URL. This functionality ensures that the evaluation results can be easily shared to others or referenced in the future, without the need of re-computation.

The requirement is satisfied if the solution stores the evaluation results and provides a unique, persistent URL for each set of results. The requirement is partially satisfied if the solution stores the evaluation result persistently (e.g. in a database), or if it provides a functionality for users to download and thereby allowing for personal persistent storage of the evaluation results, but does not provide unique, persistent URLs for each set of results. In this scenario, while users can persist the evaluation results as they want, the lack of referenceable URLs negatively impacts the ease of sharing of the evaluation results. Finally, the requirement is considered not satisfied if the solution does not provide any means of accessing the evaluation results in the future, such as through persistent URLs or download functionality.

F6 is classified at the SHOULD level because the primary goal of the solution, which is to automate WUI evaluation, can still be effectively performed without this feature. However, the capability to store and easily access past results would enhance the solution's usability as it facilitates better data management and fosters easier collaboration among users. Therefore, while not fundamental, incorporating this feature is recommended to maximize the overall value and user experience of the solution.

2.1.2 Non-functional Requirements

The following two requirements pertain to the qualities and attributes that an ideal solution should possess.

NF1 - Extension of New Metrics/Models

The solution needs to be designed with an architecture that supports extension of new metrics and models. The solution should be independently extensible, i.e. it can combine independent extensions without the need of global integrity check [28]. Ideally, it should be possible to add new metrics and models to the software system without requiring direct modification in the source code of the solution.

The requirement is satisfied if the solution fulfill all the criteria mentioned above. The requirement is considered partially satisfied if the solution demonstrates a clear effort towards enabling the extension of metrics and models, but still requires direct manipulation of the source code. In this scenario, while the solution does provide support for metrics and models extension, it necessitates additional effort as user need to have certain understanding of the codebase before they can perform extensions. Finally, the requirement is considered not satisfied if the solution offers no extension mechanisms, lacks documentation and extension points, and requires extensive modification to the source code for any new metrics or models extension.

NF1 is classified at the MUST level because easy extensibility of new metrics and models is imperative for ensuring that existing and future research findings on the relevant field, such as evaluation models and metrics, can be integrated and readily utilized by the solution.

NF2 – Accessible User Interface Design

The importance of user interface has been thoroughly addressed in the introduction chapter. In particular, ensuring the accessibility of the UI is of high importance as it directly impacts the usability of the solution for a diverse range of users, especially those with disabilities.

The UI of the solution should not contain any error as checked by an automatic accessibility checker tool. For the purpose of assessing this criterion, the accessibility checker tool WAVE (Web Accessibility Evaluation Tools) [54], will be used to analyze the solution's UI. WAVE is selected due to its ease of use and comprehensive accessibility checks. The absence of errors as identified by WAVE would indicate that the UI of the solution adheres to key accessibility standards and is free of from glaring accessibility issues.

In addition, manual inspection of the solution's UI is also necessary, as automated accessibility evaluation tools have their limitations and a number of accessibility guidelines require human judgment to be assessed [37]. Specifically, manual inspection will be performed for this following criterion: The solution should support keyboard navigation such that all functionalities that the solution offers are operable through keyboard-based interactions. As noted in WCAG 2.2 – SC 2.1.1, when content can be operated through keyboard-based interactions, it is operable by people with no or low vision as they cannot use devices which require eye-hand coordination such as computer mice effectively [27]. Other assistive technologies such as voice recognition and screen readers

also rely on keyboard accessibility. This criterion is crucial as it ensures that the solution is accessible and user-friendly for people with disabilities.

The requirement is satisfied if the solution fulfill all the criteria mentioned above. The requirement is partially satisfied when the solution support keyboard navigation as defined by WCAG 2.2 – SC 2.1.1, but there exists other accessibility error as evaluated by WAVE. Finally, the requirement is considered not satisfied when the solution does not satisfy the keyboard navigation criterion.

NF2 is classified at the SHOULD level, primarily because the intended target audience of the solution is relatively narrow, and the solution itself does not fall under the scope of products or where accessibility is a legal requirement as specified in Article 2 of the European Accessibility Act [38]. Given the specific nature of the solution, these legal mandates are not applicable. However, the inclusion of accessibility design remains a recommended practice to ensure broader inclusivity for the solution.

2.2 Related Works

This section surveys the state of the art in the field of automated user interfaces evaluation. It is organized into two parent groups:

- Metric-Based Approaches, which is subdivided into: Single-metric Approaches and Multi-metric Approaches.
- UI Evaluation Platforms.

These grouping is defined based on the focus areas and depth of evaluation that each set of tools and studies offers in the field of WUI evaluation. This categorization is designed to reflect the areas of interest of each of the state of the art discussed in this section.

The following subsections will examine corresponding works within each group and its sub-group in terms of the name and related publications, overall aim, the evaluation results, and the brief reasoning for its categorization. Works that exhibit a high degree of similarity will be collectively reviewed as a unified approach.

The assessment for works categorized under single-metric approaches will be conducted collectively, whereas works within multi-metrics approaches and UI evaluation platforms will each be assessed individually. The following symbols are used to represent the ratings according to the evaluation scheme defined in 2.1: ○ for not satisfied, ◐ for partially satisfied, and ● for satisfied.

2.2.1 Metric-Based Approaches

Metric-Based Approaches refer to a range of methodologies and software tools in the field of WUI evaluation which define and/or employ quantitative metrics to evaluate various aspects of WUI. These approaches are characterized by their use of specific, measurable, and often empirically validated parameters to objectively evaluate certain quality such as usability, aesthetics, accessibility, visual complexity, or any other key characteristics of WUIs.

This group is further subdivided into two sub-groups, differentiated based on the scope and the nature of output and/or artifact that produced by each relevant work: single-metric approaches and multi-metric approaches. In addition to these classifications, another layer of categorization based on the evaluation methodology is introduced to further stratifies the approaches discussed into distinct groups with clearer shared characteristics. Three evaluation methodology are identified:

1. **Static-code** analysis methods, which evaluate a WUI based on the analysis of its source code.
2. **Pixel-based** analysis methods, which evaluate a WUI by examining and/or extracting the key characteristics and/or features of its visually rendered interfaces (pixels).
3. **Combined** analysis methods, which involve the combination of both static-code analysis and pixel-based analysis to evaluate a WUI.

The table 2.2 provides an overview of the assessed state of the art within the group "Metric-Based Studies and Tools", showcasing the classification of each approach based on the nature of their outputs/artifacts (single-metric or multi-metric), their evaluation methodology (static code, image-based, combined).

	Single-Metric	Multi-Metric
Static code	Ngo et al. [48, 49], Ivory et al. [13]	WAVE [54], Grigera et al. [66, 67] (Kobold)
Pixel-based	Hasler and Süsstrunk [52], Reinecke et al. [50], Rosenholtz et al. [47], Mack and Oliva [79], Miniukovich & De Angeli [34], Fosco et al. [39], Xie et al. [35], Dou et al. [16], Bakaev et al. [61], Boychuk and Bakaev [43]	Zen & Vanderdonckt [51, 68, 69] (QUESTIM)
Combined	Michailidou et al. [30] (ViCRAM), Wu et al. [15], Bakaev et al. [63]	-

Table 2.2: Overview of assessed metric-based approaches with regard to the nature of output and evaluation methodology

As shown in table 2.2, most assessed approaches are pixel-based. This is because pixel-based methods, which analyze the screenshots of the UI, arguably offer a better representative of what and how users see and perceive the UI [50]. Given this distinct advantage, most recent works, and developments in the field of automated UI evaluation, as observed by the author, is now more focused on these pixel-based methods.

2.2.1.1 Single-Metric Approaches

This group is characterized by its focus on evaluating the efficacy of a specific, singular metric of the UI. A significant share of relevant research within the field focuses on in-depth analysis of an individual metric of the UI, in which the significance and methodologies for the evaluation of the metric are examined.

While the requirements defined in section 2.1 are intended for evaluating platforms that support a more comprehensive automated evaluation of WUI, this group nevertheless remains crucial to be investigated, as the insights and methodologies developed for these single-metric evaluations often serve as foundational elements for more comprehensive evaluations. For instance, AIM [14] is an example of how these singular metrics are

integrated into a unified and multi-faceted evaluation platform. It should be noted that approaches introducing multiple independent metrics, or multiple models that each only produce a singular output, are still categorized under the single-metric approaches group.

Page-level Metrics for Web Page Classification (Ivory et al)

In their study, Ivory et al. [13] investigated which page-level metrics are associated with highly rated web pages, and whether these metrics differ among different website categories. These metrics are word count, body text percentage, emphasized body text percentage, text positioning count, text cluster count, link count, page size, graphic percentage, graphics count, color count and font count. With these metrics, they created a predictive model which classify whether a web page is "good" or "bad". Their study found that most of these metrics had significant contributions to prediction of the overall quality of pages (for instance, good pages typically had a relatively lower graphic percentage). Overall, this study provides insights and potential of using simple and easy to obtain metrics to predict human judgment with some degree of accuracy.

This study is categorized within the single metric approaches since each of the metric introduced is distinct, and the developed prediction model only produces a single output, i.e. binary classification for a web page.

Colorfulness

Hasler and Süsstrunk [52] introduced a metric quantifying the colorfulness in natural images, defined as a linear combination of the mean and standard deviation of the pixel cloud in the CIELab color space. ClELab is a uniform color space including correlates of lightness, chroma, and hue [76]. The developed metric showed high correlation (95%) with the experimental data. However, hue was not considered in their experiments.

Based on this study, Reinecke et al. [50] developed a computational model for predicting perceived colorfulness of website screenshots. Their model for perceived colorfulness integrated the colorfulness metric by Hasler and Süsstrunk along with other image metrics (saturation, number of image areas, number of quadtree leaves, text area, non-text area), and 6 color percentages of pixels. As demonstrated in their experimental results, the prediction model achieved high prediction accuracy ($R^2 = .78$).

These approaches are categorized within the single metric approaches as they produce a singular output: the quantified colorfulness.

Measures of Visual Clutter

Rosenholtz et al. [47] suggested two measures of visual clutter, namely feature congestion and subband entropy. Feature congestion is based upon of extensive modeling of what makes items in a display is visually salient [47]. Color, orientation, and luminance contrast are used as features to compute a single measure of congestion clutter for the input image. Subband entropy is based on the encoding efficiency of the input image while preserving its perceptual quality [47]. In essence, it quantifies clutter based on the principle that a less cluttered image allows for more redundancy, and therefore can be encoded more efficiently. In addition to these two measures, they also evaluated another measure of clutter suggested by Mack and Oliva [79], the edge density, which is defined as the percentage of pixels that are edge pixels within the image. Their experiment results demonstrated that all three measures of visual clutter correlate well with search performance in complex imagery.

These measures are categorized as single-metric approaches because each measure is distinct, independent from the others, and require different computational methods.

Visual Complexity Measurement

Michailidou et al. [30] developed ViCRAM (Visual Complexity Rankings and Accessibility Metrics), a tool designed to quantify the visual complexity of web pages. It generates a visual complexity score based on the common aspects analysis of an HTML Document Object Model (DOM) and its visual rendering. ViCRAM visualizes the complexity of web pages in the form of a pixelated heat map which highlights the areas that are most visually complex on a web page [30]. The scores generated were validated through a user study, which demonstrated a strong correlation between ViCRAM's predicted complexity scores and user's perceptions of visual complexity.

In [15], Wu et al. presented an interdisciplinary approach for measuring the visual complexities of web pages, combining insights from the areas of HCI (i.e. ViCRAM), web mining, computer vision, and machine learning. Their method, similar first analyzes the structure of the web page, in which a number of measurable properties called features are extracted. After features extraction, machine learning algorithms are used to construct a model for measuring visual complexity. Their experimental study showed that their approach achieves better measurement performance compared to other existing methods, including the HCI-based method proposed by ViCRAM. However, it is noted that the visual feature extraction procedure in this method is highly time-consuming.

Both of these approaches, as well as any other similar ones, are categorized within the single-metric approaches as they produce a single output to represent the predicted visual complexity (pixelated heat map in ViCRAM, a scalar visual complexity score in the approach of Wu et al).

Aesthetics Metrics

Ngo et al. [48, 49] developed a computational theory for evaluating aesthetics of interfaces based on Gestalt principles. They introduced 14 formalized aesthetic measures for graphic displays, namely: symmetry, sequence, cohesion, regularity, homogeneity, rhythm, order, complexity, balance, equilibrium, unity, proportion, simplicity, density, and economy. All of the measures are focused on the formatting of the interface layout, and not on the components on the interface.

The results of their experimental study indicated that these formulae are important determinants of a system's acceptability, and they are used in many subsequent research topics, for example: models for quantification of perceived visual complexity and colorfulness by Reinecke et al. [50], evaluation of GUI aesthetics by Zen and Vanderdonckt [51, 68, 69], and aesthetic evaluation tool by Purchase et al. [31].

Separately, Miniukovich and De Angeli [34] described eight metrics to compute the aesthetics of GUI. These metrics are visual clutter, dominant colors, color range, grid quality, contrast, symmetry, congestion, and white space. These metrics were developed based on psychological investigations of what people perceive as complex and unappealing (based on [60]), as well as HCI investigations of webpage aesthetics (for example: the study by Reinecke et al. [41, 50]).

Their validation studies showed that these metrics were able to explain up to 49% of variance in the webpage visual aesthetics. However, with mobile-based GUIs, they only explained up to 32% of variance, indicating that the metrics are more suitable for evaluation of WUI than mobile-based interfaces. This finding in particular supports the decision on why the evaluation of mobile-based interfaces is out of scope for this thesis.

Both studies from Ngo et al. and Miniukovich and De Angeli are categorized within the single-metric approaches as each of the metrics that they introduced contributes to a distinct and singular assessment of a specific aspect of GUI aesthetics. Each aesthetic quality (metric) has its own formula and require, further emphasizing that the evaluation of each metric is distinct and singular in the nature.

Saliency Models

Saliency Models refer to computational models that assign a saliency value at each pixel of an image [40], aiming to predict which areas of the input image are most likely to catch human attention. One example of such model is UMSI (Unified Model of Saliency and Importance).

UMSI was introduced by Fosco et al. [39], and it is a deep learning-based model trained on natural images in addition to graphic designs. It is capable to approximate the human attention on various design classes and natural images [39]. UMSI has been shown to outperform previous state-of-the-art models, such as convolutional neural network (CNN) model for predicting saliency by Bylinskii et al. [44], the energy-based model by O'Donovan et al. [45], and the Long Short-Term Memory (LSTM)-based Saliency Attentive Model by Cornia et al. [46], according to the Pearson's correlation coefficient. The produced result of UMSI is a heatmap, where every pixel of the input image is assigned an importance value, visually representing areas of highest to lowest attention.

Given that saliency models typically generate a single, unified metric as their final output — as exemplified with UMSI, which produces a heatmap of visual importance — these models are categorized within the single-metric approaches.

Object Detection & Image Segmentation Tools

In the context of UI evaluation, object detection & image segmentation tools are software tools that analyze input UI images, in which they detect and classify the elements within the UI to facilitate the computation of other image-based metrics that rely on this analysis, such as white space proportion and grid quality [34]. One of the most notable tools in this domain is UIED (User Interface Element Detection).

UIED is an open-source toolkit for detecting GUI elements developed by Xie at al. [35]. It integrates both traditional computer vision methods and deep learning methods to perform accurate detection. This hybrid approach produces states-of-the-art detection results, outperforming both the traditional computer vision (REMAUI by Nguyen and Csallner [55] and Xianyu by Yongxin et al. [56]) deep learning methods (Faster R-CNN by Ren et al. [57], YOLOv3 by Redmon and Farhadi [58], and Centernet by Duan et al. [59]) as demonstrated by the F1-scores reported in their comparative study [35].

UIED, along with similar tools, are categorized within the single-metric approaches as they produce a singular, specific metric: the detected elements of the input UI.

Deep Learning-based Method for Aesthetics Evaluation

With the recent advancements and a surge of interests in the field of deep learning, a growing number of research has explored the use of these methods to evaluate webpage aesthetics. Webthetics, a deep neural network model developed by Dou et al. [16], is one notable example within this domain.

For the training and testing samples of the model, Webthetics uses the public dataset on visual appeal by Reinicke and Gajos [41], which contain hundreds of webpage screenshots with visual appeal ratings from nearly 40000 participants with diverse backgrounds. The experiments conducted by the study have demonstrated that the prediction results from Webthetics are highly correlated with the user ratings, achieving a Pearson value of $r = 0.85$, with $p = 0.001$. This accuracy outperformed both the state-of-the-art, hand-crafted feature-based model based on the work of Reinecke et al. [50] and previous deep learning-based method by Khani et al. [42].

Webthetics, along with similar approaches, are categorized within the single-metric approaches as they aim to predict aesthetics as a single, unified metric.

Kansei Engineering-Based ANN for Web Interface Similarity Evaluation

In [63], Bakaev et al. investigated the applicability of ANNs to predict perceived similarity assessments of user interfaces without the need of actual users. Two dimensions for similarity of WUIs are discussed: objective (UI-intrinsic) and subjective (user impressions). The respective ANN models are constructed, in which the metrics for objective model are mined from multiple sources (e.g. the source code, web analytic services such as Alexa, etc.) [71], while Kansei Engineering method is used for subjective model. Kansei Engineering [64] refers to a product development or improvement methodology which incorporates customers feelings and needs into the design process of the product.

The evaluation results of the study showed that the subjective model performed considerably better than the objective model, suggesting that subjective factors are more suitable for predicting perceived website similarity. The joint model suffered from overtraining, performed slightly worse than the subjective model in the testing and holdout dataset.

This study is categorized within the single-metric approaches because each of the prediction models is concerned with the computation of a single metric, i.e. prediction of perceived website similarity between two interfaces.

User Behavior Models for Prediction of Complexity, Aesthetics and Orderliness

In [61], Bakaev et al. explored the applicability of artificial neural network (ANN)-based user behavior models to predict the subjective assessment of WUI. 21 models were trained in total, each corresponding to a specific website domain and a particular subjective assessment dimension. For each model, the input values were 32 metrics obtained through the analysis of the websites' screenshots. Each model is then evaluated with its native testing dataset and 6 testing datasets from other domains. The results showed that prediction for other domains produced a greater mean-squared error, although the discrepancy for complexity was relatively not significant (+23%). Overall, this study provides insights into potential domain independence and reusability of user behavior models across different website domains.

This study is categorized within the single-metric approaches, as each model is dedicated to a specific website domain and focused on computing a singular, distinct WUI metric.

Entropy and Compression-based WUI Analysis

In [43], Boychuk and Bakaev investigated the applicability of Shannon's information entropy and compression-based metrics (JPEG file size and PNG file size) for predicting the visual perception of WUI.

The study found that the entropy metric was not correlated with the image size, and higher entropy led to decrease in perceived complexity and increase in perceived orderliness. Furthermore, the study demonstrated that the models incorporating the three factors (JPEG size, PNG size, information entropy) outperformed other models in predicting perceived complexity, aesthetics, and orderliness. Overall, the study demonstrated that information entropy can be a feasible metric in evaluating WUIs.

This study is categorized within the single-metric approaches, as each of the model only focus on the computation of a single metric (complexity, aesthetics, or orderliness).

Assessment of Single-Metric Approaches

Assessing the works within the single-metric approaches with the list of requirements as defined in 2.1 presents challenges. The inherent nature of single-metric approaches does not align well with most of the requirements, which are designed mainly for UI evaluation platforms. For instance, the requirement *F2 - Selectable Metrics* and *F4 - Parallel Processing of Metrics Computation* imply the availability of various metrics from which users can choose to evaluate. By definition, single-metric approaches are focused on evaluating a specific metric at a time, which makes these requirements not applicable. The requirement *NF1 - Extension of New Metrics/Models* also conflicts with the definition of single-metric approaches, rendering it not applicable.

Moreover, apart from UIED and ViCRAM, all of the discussed works in this group are not ready-to-use application; they are instead models or formulae lacking user interfaces, thereby making *NF2 - Accessible User Interface Design* largely not applicable. Since ViCRAM is not a web-based application, *NF2* is considered also out of scope for ViCRAM. For UIED, the website http://uied.online/ is evaluated. While keyboard navigation is supported for navigating the website and performing metric evaluation (as tested by the author), more than 100 accessibility errors were identified using WAVE at the time of the writing, indicating that *NF2* is only partially satisfied for UIED.

Other requirements such as *F1 - Diverse WUI Input Types Support, F5 - Parallel Processing of Evaluation Requests* and *F6 - Persistent Evaluation Results* are also not applicable for works that are not application. ViCRAM is considered to partially satisfy *F1,* as it accepts a web page (URL) as its input. On the other hand, UIED does not satisfy *F1,* as UIED can only accept screenshot as its input. Both of them do not satisfy *F5* and *F6*, as the option to evaluate multiple inputs at once and saving the evaluation result persistently are not addressed.

Evaluating the requirement *F3 - Efficient Input Pre-processing* is perhaps the most challenging, as it requires an in-depth investigation of the solution's source code. In addition, the source code for many of the works discussed in this group are not publicly available, rendering an assessment of *F3* not possible for them. However, for works in which the source code is publicly available, such as UMSI, UIED, models by Reinecke et al., and Webthetics, there appears to be no evidence of repetitive, redundant computations in the input pre-processing tasks, indicating that *F3* is fully satisfied for these works.

2.2.1.2 Multi-Metric Approaches

This group is characterized by its emphasis on evaluating a range of metrics to assess certain aspect of the UI. Unlike single-metric approaches, the models or tools within this group produce an output that represent values for multiple metrics.

Multi-metric approaches have a lot of similarities with the UI Evaluation Platforms group, as both of them emphasize more comprehensive evaluation by incorporating and computing variety of metrics. In this thesis, they are differentiated by the scope of the evaluation. Multi-metric approaches are focused on evaluating a specific dimension of the UI, such as usability, accessibility, aesthetics, or visual complexity. In contrast, UI Evaluation Platforms are designed to enable evaluation across multiple dimensions, offering a more comprehensive evaluation.

QUESTIM

QUESTIM (Quality Estimator using Metrics) is a web-based tool developed by Zen and Vanderdonckt to enable the semi-automated evaluation of UI quality based on its aesthetic-related metrics [51]. The tool requires user to manually draw the regions of interest on top of the screenshot to be evaluated. From the attributes of the regions (i.e. coordinates, dimensions, etc.), the metrics are automatically computed and presented to current user in gradient color codes (green indicates high value, red indicates low value) [68]. The metrics computed include metrics based on Ngo's formulae for aesthetic metrics, such as balance, simplicity, proportion, and symmetry, as well as newly introduced metrics such as center alignment, external alignment, and concentricity.

QUESTIM is categorized as multi-metric approaches because the application compute and integrate variety of metrics to provide a more comprehensive assessment of UI aesthetics.

QUESTIM, as claimed in [68], accepts both URL and screenshot as the UI input, which would theoretically fully satisfy *F1 - Diverse WUI Input Types Support*. However, in practice, as tested with the application's link[5], attempts to use URL as an input ended up in timeout request, suggesting that *F1* is not satisfied. This discrepancy might stem from the web application no longer being actively maintained by its developer(s). As QUESTIM automatically compute the available metrics, *F2 - Selectable Metrics* is also not satisfied. *F3 - Efficient Input Pre-processing* can be considered satisfied as QUESTIM does not

[5] http://questimapp.appspot.com/ [Retrieved: Mar 24, 2024]

perform any preprocessing task towards the UI input. Since the metrics are computed sequentially, *F4 - Parallel Processing of Metrics Computation* is not satisfied. *F5 - Parallel Processing of Evaluation Requests* is also not satisfied, as QUESTIM only allow the evaluation of one input at a time. QUESTIM partially satisfies *F6 - Persistent Evaluation Results,* as it provides the option for user to save the computed metrics locally into a file, demonstrating the tool's capability to persistently save the evaluation result, although not the ideal one.

QUESTIM does not satisfy *NF1 - Extension of New Metrics/Models,* as the application does not offer the possibility for user to add new metrics to be computed, and there is no documentation on how to add new metrics within the source code. Finally, QUESTIM does not satisfy *NF2 - Accessible User Interface Design,* as the application requires user to draw region of interests manually, which is not possible with only keyboard navigation.

F1	F2	F3	F4	F5	F6	NF1	NF2
●[6]	○	●	○	○	◐	○	○

Table 2.3: Evaluation of QUESTIM

Web Accessibility Evaluation Tools

Web accessibility evaluation tools encompass software tools designed to facilitate automated accessibility evaluation of web content by checking whether it complies with established accessibility guidelines, such as WCAG. Many web accessibility evaluation tools exist and widely used, for example: AChecker[7], Koa11y[8], Axe accessibility testing tools[9], and WAVE [54]. Among these tools, WAVE is chosen as the representative for assessment, as it is one of the most prominent tools and available to use for free as a web-based tool.[10] WAVE is developed by WebAIM at Utah State University. WAVE checks for many compliance issues found in the Section 508[11] (a federal standard for website and application accessibility in the United States) and WCAG 2.2 guidelines.

[6] If only the publication is considered
[7] https://achecks.org/checker/index.php [Retrieved: Jan 13, 2024]
[8] https://open-indy.github.io/Koa11y/ [Retrieved: Jan 13, 2024]
[9] Available at: https://www.deque.com/axe/ [Retrieved: Dec 16, 2023]
[10] Available at: https://wave.webaim.or [Retrieved: Mon 25, 2024]
[11] https://www.section508.gov/ [Retrieved: Dec 16, 2023]

After input evaluation, WAVE will present the page with embedded icons and indicators, with each presenting some information about the accessibility of the page [54]. The findings are displayed in a sidebar, where red icons indicate accessibility errors that need to be fixed, while green icons denote the accessibility features which might potentially improve the accessibility of the page. The other icons and indicators provide further information to facilitate manual accessibility and structural analysis of the page.

WAVE, along with these other web accessibility tools are categorized within the multi-metric approaches as they each evaluate a diverse number of accessibility-related metrics.

Since WAVE requires a URL as its input, *F1 - Diverse WUI Input Types Support* is considered partially satisfied. After WAVE receives the input, it will automatically "compute" the accessibility metrics for the input. This means that *F2 - Selectable Metrics* is also not satisfied, as user does not have control on which metrics to be computed. *F3 - Efficient Input Pre-processing* is considered satisfied as WAVE evaluate the input based on the source code analysis, eliminating the need of additional pre-processing tasks for its input. Without access to its source code, it is not possible to ascertain whether WAVE satisfies *F4 - Parallel Processing of Metrics Computation* or not. However, given the relatively simple and computationally fast tendency of source code analysis methods, it is probable that the accessibility rules are checked sequentially, which would make *F4* not satisfied. WAVE also does not satisfy *F5 - Parallel Processing of Evaluation Requests,* since it can only accept a single URL at a time as its input. *F6 - Persistent Evaluation Results* is also not satisfied, as WAVE lacks the functionality to store the evaluation results.

WAVE does not satisfy *NF1 - Extension of New Metrics/Models,* as it does not provide user the ability to introduce new rules (metrics) to be evaluated. Conversely, WAVE fully satisfy *NF2 - Accessible User Interface Design*, since keyboard navigation is fully supported, as well as absence of accessibility errors in its web application as verified by WAVE itself.

F1	F2	F3	F4	F5	F6	NF1	NF2
◐	○	●	○	○	○	○	●

Table 2.4: Evaluation of WAVE

Kobold

Kobold is an open-source tool[12] for detecting usability problems through real user interaction events, developed by Grigera et al. [66]. Kobold works as follows: first, it mines and collecting relevant UI events in the client. Then, the usability events are analyzed to detect potential usability problems. Finally, the tool recommends usability refactoring to solve the detected problems. In some cases, these refactoring can be applied automatically or semi-automatically with some input from users [66]. Overall, there are 16 kinds of usability events defined, detailed in their previous work [67].

Their validation experiment showed that the tool found half of the problems that the manual tests had found [67]. It also demonstrated that the websites refactored with Kobold scored higher compared to the original sites in the measured aspects of satisfaction (obtained from questionnaire), efficiency (average time taken to complete the task) and effectiveness (task completion percentage).

Kobold is categorized within the multi-metric approaches as the platform analyze variety of usability smells, each of which can be considered a usability-related UI metric.

In order to evaluate a website using Kobold, users are required to register an account, and the tool will provide them with a JavaScript snippet to embed in their application's header [66]. This implies that Kobold can only be used with web applications where the user has access to the source code. Consequently, *F1 - Diverse WUI Input Types Support* is not satisfied, as Kobold lacks the capability to accept and evaluate arbitrary URLs. *F2 - Selectable Metrics* is also not satisfied, as users do not have control over which usability events are evaluated. *F3 - Efficient Input Pre-processing* is considered satisfied as Kobold does not perform pre-processing tasks for its input. It is described in [67] that each application under analysis has its own set of usability smells finders, and in order to produce results in real time, usability events are processed at the moment they arrive in the server, such that each of the individual finder re-evaluates the events to detect potential new usability smells, suggesting that computations for each metric (in this case, usability smells) are running independently and in parallel. This would mean that *F4 - Parallel Processing of Metrics Computation* is satisfied. *F5 - Parallel Processing of Evaluation Requests* is not satisfied, as Kobold does not address the possibility of evaluating multiple inputs at a time. As each evaluated website require prior registration and subsequently have a

[12] Source code available at: https://github.com/lifia-unlp/Kobold [Retrieved: Jan 17, 2024]

unique URL which include information on the detected usability smells (i.e. the evaluation results) of the website, *F6 - Persistent Evaluation Results* is considered satisfied.

It is described in [67] that the architecture of the platform is designed to allow easy extension for the set of finders. This demonstrates a certain level of extensibility, albeit requiring direct source code manipulation, which means that *NF1 - Extension of New Metrics/Models* is only partially satisfied. Finally, *NF2 - Accessible User Interface Design* is considered not satisfied as an element of the home page (register button) is not accessible with keyboard navigation.

F1	F2	F3	F4	F5	F6	NF1	NF2
○	○	●	●	○	●	◐	○

Table 2.5: Evaluation of Kobold

2.2.2 UI Evaluation Platform

This group is characterized by its focus on enabling the computation of multiple metrics from diverse dimensions of UI. On the contrary with approaches belonging to another category, the works within this group do not focus on defining new metrics or constructing new models for UI evaluation. Instead, they aim to integrate existing metrics/models into a single, unified platform to facilitate an easier use of the metrics that are dispersed across different research studies. The works within this group are the most similar with what this thesis aim to achieve.

WUI Measurement Platform

Bakaev et al. [62] developed an ontology-based integration platform for assessing WUI capable of working with various remote services to enable using WUI metrics supplied by different providers. The platform operates by sending a WUI screenshot or website URL to a remote service using its supported methods and protocols. The platform then receives the metrics and other output and store them in the platform's database.

In addition to the visual analyzer tool developed by Bakaev et al. (described in [62]), AIM [14] is another remote service used for providing additional WUI metrics. Example of these metrics include JPEG file size, proportion of whitespace, among others. Notably, the WUI measurement platform is designed to accommodate both code-based (static)

and image-based WUI analysis metrics, allowing for a comprehensive and extensible approach to WUI assessment.

WUI Measurement Platform is categorized within the UI Evaluation Platform group as it is capable of computing WUI metrics supplied by different providers.

Since WUI Measurement Platform accepts both URL and WUI screenshot as its input, *F1 - Diverse WUI Input Types Support* is satisfied. *F2 - Selectable Metrics* is also satisfied as users can select/deselect which metrics to evaluate. As described, the platform works by sending the input to various remote services, each responsible for computing different metrics. This setup implies that the same preprocessing tasks might be performed by each service. Hence, *F3 - Efficient Input Pre-processing* is considered not satisfied due to the potential of repetition of same preprocessing tasks. *F4 - Parallel Processing of Metrics Computation* is also not satisfied, as the metrics computation is dependent on AIM, which compute metrics sequentially. Since the platform can only accept one input at a time, *F5 - Parallel Processing of Evaluation Requests* is also not satisfied. Considering that the evaluation result is saved in a database, as noted in [62], suggest that the results are preserved for future reference. However, as it is currently not referenceable via a URL, *F6 - Persistent Evaluation Results* is considered partially satisfied.

Given the platform's design, which works with various remote services to facilitate the use of metrics from different platforms, *NF1 - Extension of New Metrics/Models* is considered partially satisfied, as the architecture technically supports addition of metrics/models, albeit would require direct modification of the platform's source code. *NF2 - Accessible User Interface Design* is considered partially satisfied as keyboard navigation is supported for navigating the website[13] and performing metric evaluation as tested by the author, but there exist several accessibility errors as evaluated by WAVE.

F1	F2	F3	F4	F5	F6	NF1	NF2
●	●	○	○	○	◐	◐	◐

Table 2.6: Evaluation of WUI Measurement Platform

[13] http://va.wuikb.info/ [Retrieved: Feb 7, 2024]

AIM

AIM (Aalto Interface Metrics) is a web-based service developed by Oulasvirta et al. [14] which enable the computation of numerous empirically validated web metrics and model from various research studies. In total, 26 metrics are available for selection, with the majority of the metrics are focusing on visual characteristics. As the time of the writing, they cover 4 categories: 1 metric accessibility, 1 metric for aesthetics, 12 metrics for color perception, and 12 for perceptual fluency. Example of the metrics integrated include colorfulness by Hasler and Süsstrunk [52], Unified Model of Saliency and Importance by Fosco et al. [39], UIED segmentation by Xie et al. [35], and various interface metrics described in [34] by Miniukovich and De Angeli. For all metrics, the information on the references, overview on what the metric does, as well as an indicator of empirical evidence for its predictive power and the relevance are provided [14].

AIM is categorized within the UI Evaluation Platform group as it facilitates the computation of numerous metrics which are derived from various existing research works, encompassing multiple dimensions of UI.

Limitations of AIM are already mentioned in 1.2, which is described to provide an illustrative example that underscores the motivation of the thesis. They are as follows: AIM only accepts one input at a time, the evaluation results are not persistently saved, the computation of selected metric is performed sequentially, and despite the platform is designed with extensibility in mind, directly changing the source code is necessary to add new metrics. This means that the requirements *F4 - Parallel Processing of Metrics Computation*, *F5 - Parallel Processing of Evaluation Requests*, and *F6 - Persistent Evaluation Results* are not satisfied, while *NF1 - Extension of New Metrics/Models* is partially satisfied.

Since AIM accepts both URL and PNG image as input, *F1 - Diverse WUI Input Types Support* is satisfied. *F2 - Selectable Metrics* is also satisfied, as users can select/deselect which metrics they want to evaluate. However, some redundancies are observed with AIM's preprocessing operations. For instance, multiple metric implementations (feature congestion and subband entropy) redundantly perform the conversion of WUI images to the CIELAB color space. Consequently, if multiple selected metrics require this conversion, it would be executed more than once. On the other hand, for image segmentation, it is optimized to be performed at most once per WUI input, and the results then utilized in any subsequent metric computation steps that need them. Thus, AIM only partially fulfills *F3 - Efficient Input Pre-processing*, as it demonstrates optimization in certain preprocessing areas while redundancy persists in others. Finally, while keyboard

navigation is supported for the entire process of providing input, selecting metrics, and evaluating the WUI, there exists some accessibility errors as evaluated by WAVE, which means that *NF2 - Accessible User Interface Design* is only partially satisfied.

F1	F2	F3	F4	F5	F6	NF1	NF2
●	●	◐	○	○	○	◐	◐

Table 2.7: Evaluation of AIM

2.3 Analysis of Assessment Results

The state of the arts of the field automated UI evaluation have been examined and evaluated in section 2.2, in which the works categorized within single-metric approaches are assessed collectively, while works categorized within multi-metric approaches and UI evaluation platforms are assessed individually. In this section, the assessment results are to be analyzed, and the limitations of existing works will then be derived based on the analysis.

2.3.1 Analysis of Assessment Results by Requirements

In this section, the assessment results from all examined works are analyzed with regard to list of requirements defined in 2.1. Table 2.9 below provides an overview of the assessment results. Apart from ViCRAM and UIED, single-metric approaches are grouped into one column as they share the same assessment results.

Approach	F1	F2	F3	F4	F5	F6	NF1	NF2
ViCRAM	◐	○	●	○	○	○	○	○
UIED	○	○	●	○	○	○	○	◐
Other Single-metric Approaches	○	○	●	○	○	○	○	○
QUESTIM	●	○	●	○	○	◐	○	○
WAVE	◐	○	●	○	○	○	○	●
Kobold	○	○	●	●	○	●	◐	○
WUI Measurement Platform	●	●	○	○	○	◐	◐	◐
AIM	●	●	◐	○	○	○	◐	◐

Table 2.8: Assessment results overview (F1: Diverse WUI Input Types Support, F2: Selectable Metrics, F3: Efficient Input Preprocessing, F4: Parallel Processing of Metrics Computation, F5: Parallel Processing of Evaluation Requests, F6: Persistent Evaluation Results, NF1: Extension of New Metrics/Models, NF2: Accessible User Interface Design)

F1 – Diverse WUI Input Types Support

Given that *F1* assumes that the assessed work is an application, this requirement is not applicable to almost all of the works within single-metric approaches, with only ViCRAM partially satisfying the criteria. Among works in other groups, the fulfillment level is quite high. Specifically, QUESTIM and the two discussed works in UI evaluation platforms group fully satisfy the criteria. This demonstrates that more relevant existing state of the art, i.e. those with objectives more closely aligned with this thesis, have taken this requirement into account during the development of their respective solutions.

F2 - Selectable Metrics

F2 is not applicable with single-metric approaches, as this requirement implies there are multiple metrics available for selection and computation. Among works in other groups, only AIM and WUI Measurement Platform fully satisfies this requirement. This suggests that most existing works have not yet reach the state of maturity where enhancing user experience through flexibility in selecting metrics to compute is prioritized. Alternatively, for some other works, the metrics computed are either too limited in number or computationally simple, leading to the perception that fulfilling *F2* is unnecessary.

F3 - Efficient Input Pre-processing

The majority of existing works observed either do not require input preprocessing to perform the metric computation or do not perform input preprocessing tasks effectively. As observed, both AIM and WUI Measurement Platform has the potential of performing repetitive preprocessing tasks. Among works in other groups, F3 is considered fully satisfied mainly due to the lack of clear input preprocessing tasks, not because they are optimized.

This suggests that fully satisfying *F3* becomes harder with more complex solutions. Furthermore, some preprocessing tasks might be considered to be to computationally simple to optimize (e.g. conversion to RGB with AIM), and it is not worth the effort to optimize as it would potentially lead to a more complicated workflow for the metrics computations.

F4 - Parallel Processing of Metrics Computation

Similar with *F2*, *F4* implies that there are multiple metrics available for selection and computation, and hence it is not applicable for single-metrics approaches. With sole exception of Kobold, which only focuses on usability-related metrics, all other existing

works assessed do not fulfill the requirement. This indicates that the potential of improving the time performance of multiple metrics computation has not yet been explored within the field of automated UI evaluation. As observed by the author, most works seem to be focused on the accuracy and relevance of the computed metrics.

F5 - Parallel Processing of Evaluation Requests

Notably, none of the assessed works satisfy *F5.* For instance, AIM is designed to only handle one input at a time, i.e. it can only compute the metrics of a single input per session. This zero-fulfillment level is within author's expectations, considering that *F5* is a SHOULD-level requirement. It mainly concerns on improving user experience and is not part of the core functionality essential for automated UI evaluation tools.

F6 - Persistent Evaluation Results

Among the assessed works, only Kobold fully satisfies *F6,* while both QUESTIM and WUI Measurement Platform partially satisfy the requirement. However, benefits of partial satisfaction are limited given the lack of referenceable URL for the stored results, rendering them not easily sharable. Like *F5*, *F6* is classified as a SHOULD-level requirement. This assessment result underscores that most existing works prioritize core evaluation capabilities, often overlooking features that can improve the user experience and efficiency of the platform.

NF1 - Extension of New Metrics/Models

NF1 is not applicable to single-metric approaches due to the conflicting nature of the requirement and group definition. Among works in other groups, Kobold and both of the assessed UI Evaluation Platforms partially satisfy the requirement. This suggests that the importance of extensibility is acknowledged within the field. However, creation of a system that is extensible without requiring direct source code manipulation is a complex task, and not yet addressed within the existing evaluation platforms.

NF2 – Accessible User Interface Design

As *NF2* is concerned with the user interface of the solution, it is not applicable to all of the works within the single-metric approaches with sole exception of UIED. Notably, only WAVE fully satisfies the requirement, while UIED and AIM only partially satisfy the requirement. Given that *NF2* is a SHOULD-level requirement and does not affect the evaluation process for most of the target audience, this low fulfillment level aligns well with author's expectations.

2.3.2 Limitations of Existing Works

The majority of existing works within the field fall under the single-metric approaches, which generally have differing objectives with what this thesis has, as demonstrated by how most of them failing to fulfill most of the requirements established in 2.1.

With works from other categories, it is observed that each of them demonstrates varying degrees of compliance, with some requirements either partially or fully satisfied. However, none fully address all of the listed requirements. Notably, AIM, the assessed platform which fulfills the most requirements, still exhibits considerable shortcomings, as it does not satisfy *F4 - Parallel Processing of Metrics Computation, F5 - Parallel Processing of Evaluation Requests,* and *F6 - Persistent Evaluation Results,* and only partially satisfies *NF1 - Extension of New Metrics/Models* and *NF2 – Accessible User Interface Design*.

Overall, it is observed that there exists only a limited number of relevant works that share similar focus and objective to this thesis. Among these, the primary emphasis appears to be on the core functionalities, while functionalities or features to improve user experience and efficiency of the platform are overlooked. As illustration, none of the works assessed satisfy or even partially satisfy *F5* and *F6*, which both are SHOULD level requirements. Furthermore, beyond basic compliance (i.e. keyboard navigation), accessible user interface design does not seem to be a significant consideration in most existing works.

Moreover, the absence of platforms which allow for extension of new metrics / models without directly manipulating the platform's source code highlights a gap in the field. Parallel metrics computation is also mostly not considered, with most works either only compute a single metric or multiple metrics sequentially. These limitations collectively provide a compelling reason for the development of more efficient, user centric and adaptable WUI evaluation platform, as it would represent a significant advancement within the research domain.

2.4 Summary

This chapter has elicited a set of requirements based on the motivation, research problems, and challenges outlined in the introduction chapter. Existing state of the arts in the field of automated user interfaces evaluation are examined and assessed according to the elicited requirements. A grouping scheme was introduced to categorize the examined works, and the analysis of the assessment results are presented. The limitations of existing works are identified, i.e. the lack of focus on functionalities and features aimed for improving user experience and efficiency, parallel metrics computation, as well as extensibility without source code modification. These shortcomings underscore both the relevance and necessity for the solution presented in the following chapter. The findings and insights obtained from this chapter lay the groundwork for developing the concept for the proposed solution.

3 Concept

This chapter outlines the conceptual framework of the proposed solution aimed to address the shortcomings of existing works. As identified in 2.3.2, these following gaps exist within the research field:

- Focus on functionalities or features to improve user experience and efficiency of the solution, e.g. the capability to process multiple evaluation requests at a same time, the capability to persistently store the evaluation results each with a unique, referenceable URL, and accessible user interface.
- Extensible evaluation platform which does not rely on availability of the platform's source code.
- Parallel metrics computation.

In order to address these shortcomings, a web-based platform satisfying the requirements detailed in 2.1 is proposed as the solution. The concept of the software architecture and the workflow, as well as alternatives approaches with regard to the aspects of the concept are to be presented within this chapter. This chapter outlines the conceptual framework of the proposed solution aimed to address the shortcomings of existing works. As identified in 2.3.2, these following gaps exist within the research field:

- Focus on functionalities or features to improve user experience and efficiency of the solution, e.g. the capability to process multiple evaluation requests at a same time, the capability to persistently store the evaluation results each with a unique, referenceable URL, and accessible user interface.
- Extensible evaluation platform which does not rely on availability of the platform's source code.
- Parallel metrics computation.

In order to address these shortcomings, a web-based platform satisfying the requirements detailed in 2.1 is proposed as the solution. The concept of the software architecture and the workflow, as well as alternatives approaches with regard to the aspects of the concept are to be presented within this chapter.

This chapter outlines the conceptual framework of the proposed solution aimed to address the shortcomings of existing works. As identified in 2.3.2, these following gaps exist within the research field:

- Focus on functionalities or features to improve user experience and efficiency of the solution, e.g. the capability to process multiple evaluation requests at a same time, the capability to persistently store the evaluation results each with a unique, referenceable URL, and accessible user interface.
- Extensible evaluation platform which does not rely on availability of the platform's source code.
- Parallel metrics computation.

In order to address these shortcomings, a web-based platform satisfying the requirements detailed in 2.1 is proposed as the solution. The concept of the software architecture and the workflow, as well as alternatives approaches with regard to the aspects of the concept are to be presented within this chapter.

3.1 Software Architecture

The proposed solution in this thesis will be a web-based application named **WUI Auto Evaluate**. Web-based application is preferred over traditional desktop application as it offer significant advantages, namely:

- Instant worldwide deployment with the world wide web (WWW) [20, 21, 22]
- Requiring no installation or manual upgrades as the web browser is used as the runtime environment. [20, 21]
- High level of interactivity. [22]

Furthermore, the requirement *NF2 – Accessible User Interface Design* can only be fully satisfied when the solution is web-based, since web accessibility evaluation tools such as WAVE are specifically tailored for evaluating web contents. Bearing the advantages and the requirement in mind, the design of the software architecture for WUI Auto Evaluate will solely focus on the creation of a suitable web-based solution.

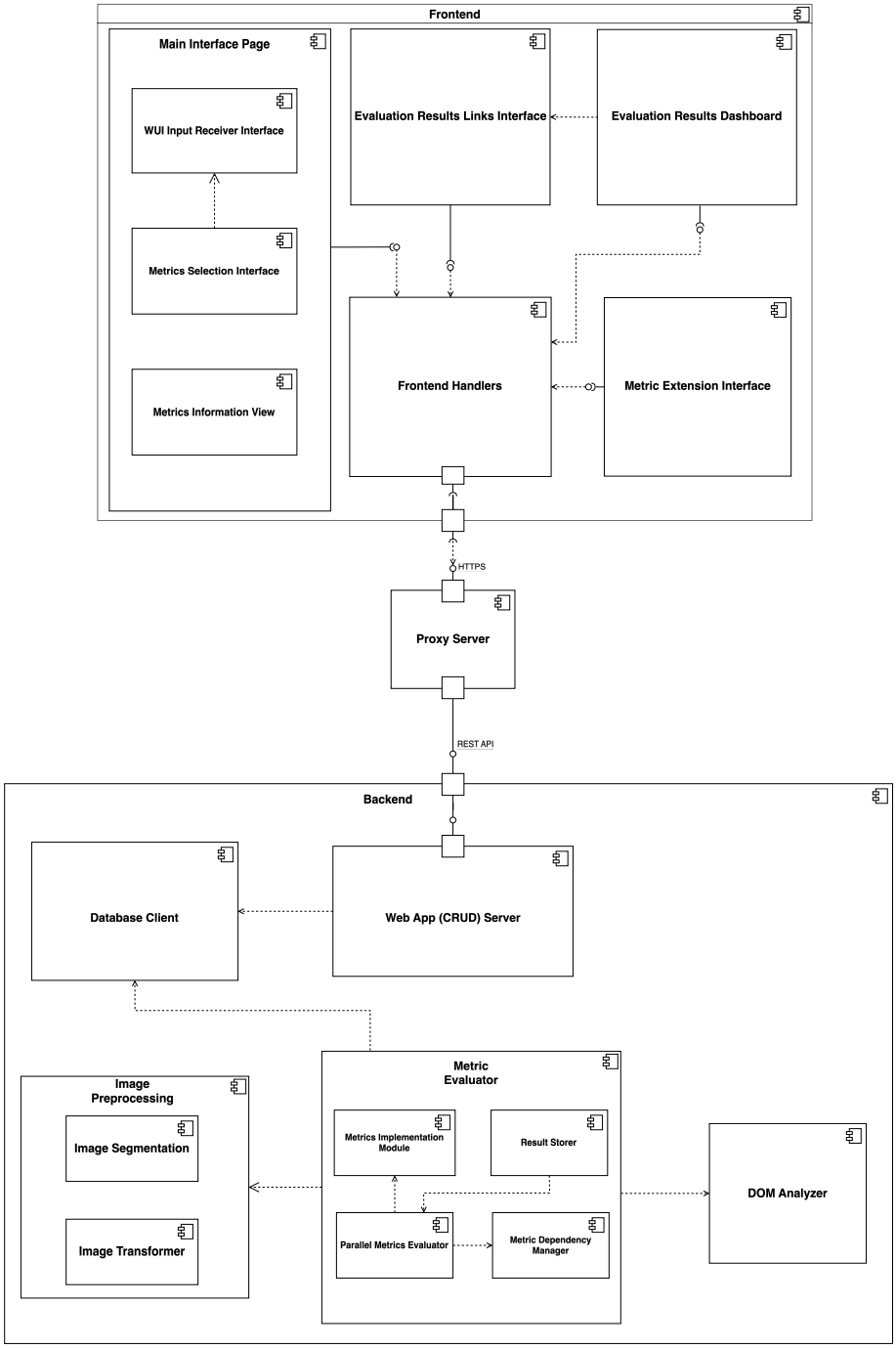


Figure 3.1: Component Diagram of the Software Architecture Overview

Figure 3.1 shows a Unified Modeling Language (UML) [75] component diagram illustrating the overview of the planned software architecture for WUI Auto Evaluate. For the sake of aesthetics, the database is omitted from the diagram. Nonetheless, the existence of the Database Client component within the backend implicitly suggests that there is an underlying database with which the backend interacts. In addition, while not explicitly shown in the figure, the software architecture allows for multiple backend instances.

WUI Auto Evaluate comprises two main components: the frontend and the backend. The frontend is responsible for the user interface, i.e. receiving inputs and presenting the evaluation results to users. The backend is responsible for handling any processes necessary for computing the metrics and communicating with the database as needed. In addition, a proxy server is integrated into the architecture to manage and route incoming requests to appropriate backend server instances. The following subsections will examine the components and their interactions within the overall framework in further detail.

3.1.1 Frontend

The frontend layer is composed of 4 components responsible for the overall user interface of the platform and one additional component, the Frontend Handlers, which manages the communication with the server and the global state (state shared across multiple components) of the frontend. The interface components are designed to be self-contained, each responsible for handling its own state management. This approach to state management, encompassing both global and local scopes, is adopted to enable the frontend to quickly and dynamically respond to user actions that require no server interaction.

Frontend Handlers

This component is introduced to centralize the management of the communication with the server (i.e. handling of API calls and responses) as well as global client state management. Interface components will interact exclusively with the handlers rather than making direct API calls. By introducing this form of abstraction, the complexity of the API communication is encapsulated within one component, allowing UI-related components to focus on their primary functionalities (e.g. visual presentation and user interactions).

Main Interface Page

The Main Interface Page serves as the primary page of the platform. It is responsible for receiving the user inputs necessary for evaluating web user interfaces. This encompasses the WUIs to be analyzed and the corresponding metrics to be computed. Initially, the user is presented with an interface to input the WUI that they want to evaluate. To facilitate simultaneous evaluation of multiple WUIs (i.e. to satisfy *F5 - Parallel Processing of Evaluation Requests*), the interface of WUI Auto Evaluate is designed to be able to accept multiple WUI inputs at once.

Two options for input submission are provided: User can either manually enter and add a URL to the list of inputs or upload WUI file(s) directly from their device. For file upload, two file formats are accepted: HTML and PNG. Additionally, a URL validation mechanism is incorporated. If a user tries to add a URL that violates the platform's recognized valid syntax of URL, an error indicator will be displayed, and the respective URL input will not be added to the list of inputs. Once one or more WUI are inputted, users can confirm their WUI inputs. Upon confirmation, the user is brought to the next step: selecting the metrics to compute for each input, where one metric at minimum must be selected for each input. Otherwise, the trigger to evaluate the confirmed WUI inputs is disabled. Users also have the option to return to the previous step if needed.

Certain metrics are not available for inputs with certain types. For instance, metrics that require DOM access are not available for inputs in PNG file format, and consequently, will not be presented for selection for these inputs. Apart from this specific input type requirement, all metrics will be in general selectable and deselectable.

This approach of separating the processes of WUI input and metric selection is chosen over combining them into a single, comprehensive procedure, to prevent visual cluttering and cognitive overload to the user due to excessive information and options.

Alongside the interface for WUI input and metric selection, the main interface page also contains a section presenting the information on metrics available for selection. This information includes the names of the metrics, their brief descriptions, the file formats they accept, as well as accessible hyperlinks to their respective references. This ensures that users are sufficiently informed on the nature of the metrics and offers some reading materials for those who are interested to deepen their understanding on certain metrics. Once users confirm their inputs, they are navigated to the Evaluation Result Links Interface.

Evaluation Results Links Interface

The Evaluation Results Links Interface serves as an intermediary which presents the URL(s) referencing to the evaluation results of each input WUI. A table is displayed, with following information provided for every assessed WUI: its name, its format type, and the unique URL that directs to the evaluation results of that WUI. The table is dynamically updated whenever new evaluations results links become available. A visual indicator is presented to show how many remaining evaluation results links are not yet ready. Additionally, any instances of request errors will be informed to users in this interface. This page is introduced as each of the WUI evaluation result(s) needs to be stored in a unique URL for reference, and it is not feasible to display all of the evaluation results at once when high number of WUIs are evaluated at once.

Evaluation Results Dashboard

The Evaluation Results Dashboard is responsible for presenting the evaluation result of the input in the user interface. Each URL will provide near real-time updates on the computation progress for the selected metrics, such that the user interface is updated as soon as a result for a specific metric becomes available.

Initially, the dashboard will display the selected metric(s) for evaluation and the indicator for each metric on whether the evaluation result is ready or still being computed. In addition, the WUI input screenshot will be available for preview. This screenshot image is hidable, and clicking on it would lead open a new tab to the URL where the image, in its intrinsic size, is stored. An option to copy the current URL to the clipboard is also provided for user's convenience.

After all metric computations are completed and their results visualized, the URL achieves its final stable state, i.e. the user interface for this URL will no longer undergo changes. Similar to how the evaluation result is presented in AIM, each metric evaluation result will be accompanied with a brief description of the metric and clickable links to its references. When applicable, a description and interpretation to the value of the metric evaluation result will also be presented to the user. There are three possible formats for the value of each evaluation result: text (with numbers also categorized as text), a JavaScript Object Notation (JSON) file, or a PNG image. When the result is a JSON, a download button for the file is presented to users. When the result is an image, similar with the WUI input screenshot preview, the image is clickable, and clicking it would direct users to the URL where the image is hosted in its intrinsic size.

Metric Extension Interface

The Metric Extension Interface is responsible for facilitating the addition of new WUI metrics within WUI Auto Evaluate. This page will enable users to propose a new metric through uploading the metric implementation file with a specific format. Alongside the metric implementation file, users are required to submit other metadata related to the proposed metric.

This includes the metric name, metric description, metric's accepted input type, a specific file listing the dependencies required by the metric, image preprocessing tasks required for the metric evaluation, pertinent references, expected result(s) of the metric computation, and download links to the corresponding model(s) if there is any. In this context, a "model" refers to any external code file that is necessary for the metric to perform its computations but cannot be included directly within the metric implementation file itself.

In addition, users have the option to provide their email addresses via the interface, enabling direct communication to inform them whether their metric(s) is approved or not approved, and in this case along with the reason(s) why. Detailed specifications on these file format requirements will be described in the next chapter on the implementation details. After the file is uploaded, each extension will be reviewed manually for approval. Only approved metrics will be integrated into WUI Auto Evaluate. The integration of new metrics is to be handled manually by the approver.

This approach builds on the concept implemented in AIM, in which new metrics that meet specific format criteria can be registered into the platform manually. However, unlike AIM, where contributors are required to directly modify the source code, this approach aims to enable the extension of new metrics and models without requiring contributors to have direct access to or modify the platform's underlying source code by themselves. This method simplifies the process for contributors, allowing them to focus on the development and validation of their metrics and models rather than navigating the complexities of the system's codebase. Abstracting the integration process ensures that the platform remains secure and stable while still fostering innovation and expansion through community contributions.

3.1.2 Backend

The backend layer is responsible for the core operational processes of WUI Auto Evaluate. Its responsibilities include processing and analyzing the inputs, computing the metrics, handling their dependencies, managing the integration of new metrics and models into the system, and ensuring that the evaluation results are stored persistently. It is composed of several components, each with a specific role to achieve the functionalities of the platform.

Web App Server

The Web App Server is responsible for handling the requests coming from the frontend. This encompasses initial processing of the request and transmitting it to the relevant components, i.e. to the Database Client for storing the results of initial processing of the inputs and the Metric Evaluator for computing the metrics. The server operates asynchronously with the Metric Evaluator; it does not wait for the completion of metric computations. Once it receives a response from the Database Client, the server processes this information and relays it to the frontend. For each response, the following information will be sent to the frontend: the name of the WUI, the type of the WUI (URL, PNG, or HTML), and a unique identifier that corresponds to the URL referencing to where the WUI's evaluation results are persistently stored.

Database Client

The database client is composed of a set of methods responsible for managing all the communications between the backend layer and the database. Details on the concept of the database is described in the section 3.1.3.

Screenshot Capturer

The Screenshot Capturer is responsible for capturing the screenshot of a WUI input, applicable when the input is a URL or an HTML file. To satisfy *F3 - Efficient Input Preprocessing*, this process will only be performed once for each WUI input. The resulting output is then made available within the entire WUI evaluation process and passed to other components for further analysis of the input as needed.

Before capturing the screenshot, the capturer will wait briefly for the input content to fully load, as dynamic content can take additional time to appear. This is achieved by

monitoring the "document.readyState"[14] property of the input for a signal that the load is complete. In addition, a wait of 5 seconds is set as according to a 2023 survey examining the page load times of the top 100 websites worldwide [77], the average page load time is 2.5 seconds on desktop. This threshold ensures that most pages will load completely while preventing excessive wait times that would negatively affect user experience.

The Screenshot Capturer will take the screenshots and store them in a specific format: as a PNG image with RGB color model and image size of 2400x2400 pixels (window size = 1200x1200 pixels, device pixel ratio = 2). This relatively large window size is selected to allow the capturer to capture a comprehensive view of the URL's content while maintaining the quality of the screenshot.

Image Processing

The Image Processing component is responsible for handling the preprocessing for image-based evaluation tasks within the platform. This component is comprised of two sub-components, the Image Segmentation component, and the Image Transformer component. The Image Segmentation component is responsible in analyzing and segmenting the WUI, outputting a list of detected elements and their associated data. The Image Transformer component is responsible for transforming the input image into suitable format(s) required for the computation of the selected metrics. These include conversion to grayscale, conversion to JPEG image file format, and conversion to CIELAB color space. The CIELAB image format is required for the computation of metrics related to the measures of visual clutter [34, 47].

For each evaluation request, each of the necessary preprocessing task will be executed only once, and the result is then made available for use in all subsequent metric computation steps that depend on it, thereby optimizing efficiency by avoiding redundant operations for the same input, and ultimately fulfilling *F3 - Efficient Input Pre-processing*.

DOM Analyzer

DOM Analyzer is responsible for extracting the rendered DOM of a WUI input and subsequently parsing and retrieving useful information from it. DOM is a programming API for web documents, and it represents the logical structure of the document modeled using objects [72]. This component is only relevant for WUI input with URL or HTML

[14] https://developer.mozilla.org/en-US/docs/Web/API/Document/readyState [Retrieved: Mar 21, 2024]

format. This extraction and analysis process will be executed only once per WUI input and initiated only if at least one of the metrics selected for evaluation requires information from the DOM of the input. The results of this process are subsequently made available for the Metrics Evaluator for the computation of the relevant metric(s). The result of the DOM Analyzer will include the parsed visible text content of the WUI and information on each of the DOM element contained within the WUI. The listing 3.1.2 illustrates the planned format for the DOM analysis output.

```
{
  "text": "visible text",
  "elements": [
    {
      "tag": "html",
      "attributes": {},
      "children": [ "head", "body" ]
    },
    {
      "tag": "a",
      "attributes": {
        "href": "FAQ/List.html"
      },
      "children": []
    }
  ]
}
```

Listing 3.1.2: Example of the DOM Analyzer output

Metric Evaluator

Metric Evaluator is responsible for the computation of all metrics available within the platform. It is composed of 4 sub-components: Metric Implementation Module, Metric Dependency Manager, Parallel Metrics Evaluator, and Result Storer.

Metric Implementation Modules comprise a collection of classes, each responsible for the computation of a single, specific metric. Every metric implementation will adhere to a predefined metric interface definition. This is required to ensure that the implementations can be integrated within the system's architecture.

Metric Dependency Manager is responsible to dynamically load the implementations of the selected metrics, along with their dependencies. It also determines the required preprocessing tasks (e.g. image segmentation, DOM analysis, etc.) based on the selected metrics. These tasks are executed once, with the results made available to any metric

implementations requiring them. After metrics are loaded to the system and their associated preprocessing tasks completed, the Metric Dependency Manager communicates with Parallel Metrics Evaluator, which then performs parallel computations of the selected metrics. Process-based parallelism is to be implemented for the metrics computation. Process-based parallelism is a form of parallelism where multiple processes are spawned and utilized to perform tasks in parallel.

This approach of parallelism is preferred over thread-based parallelism as it enables the server to leverage multiple processors [73]. Purely thread-based parallelism, on the other hand, shares only one memory space among all threads [74], which can potentially lead to bottlenecks for resource-intensive tasks. Furthermore, certain programming languages such as Python has a built-in global interpreter lock[15] mechanism which prevent multiple threads to execute simultaneously, thus making multithreading to not really perform tasks in parallel in those programming languages, and thereby not suitable to fulfill the defined requirement. Hybrid approach of combining both multiprocessing and multithreading was also considered, but due to relatively limited availability of supporting libraries and unclear performance benefits, it is ultimately not the selected approach for this platform.

For each selected metric, a process is created, and these processes are then executed simultaneously. Upon completion of a metric computation, the corresponding result is forwarded to Result Storer. This component subsequently processes the result further and transmit it to the database client, which then store the result in the database.

3.1.3 Database

The database is responsible for storing any relevant artifacts that requires persistence. These include the screenshots of the WUI, the metadata of the WUI, and the evaluation results of the WUI. Figure 3.2 shows the database schema for enabling the persistence of the evaluation results.

[15] https://docs.python.org/3/glossary.html#term-global-interpreter-lock [Retrieved: Feb 29, 2024]

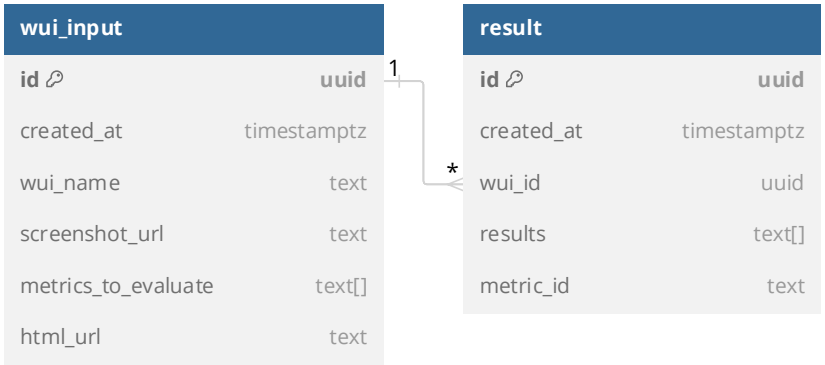


Figure 3.2: Database Schema for Persisting WUI Evaluation Results

Two tables are utilized, one for storing the WUI input metadata and another for storing the evaluation results. The primary key of both tables, *id*, use Universally Unique Identifier (UUID) version 4 [78] as their type. The value for the *id* and *created_at* columns are generated automatically by the database when a record is inserted to the table.

In the **wui_input** table, the *metrics_to_evaluate* column contains textual information for which metrics to be evaluated for the corresponding WUI. The *screenshot_url* column stores the URL of the screenshot of the WUI, and the *html_url* column stores the URL to the HTML file if the WUI input is of type HTML. Otherwise, the value of the *html_url* will be null.

In the **result** table, the *wui_id* serves as a foreign key that links each entry to its corresponding WUI input. The *metric_id* column stores an identifier corresponding to the evaluated metric, and it is used to construct a unique URL for accessing the evaluation results associated with this WUI input. The *results* column stores the evaluation results for a particular WUI and metric. This includes textual data for standard results and, in cases where the result is an image, the URL pointing to the image. Files are stored in a separate object storage system, as the process of saving and retrieval of binary data in a database are generally much slower and would increase the size of database dramatically, which could potentially lead to serious performance issues.

A separate table is used to stores the information metrics requested for extension. This information will include the URLs pointing to where the metric implementation file, file containing the list of metric's required dependencies, and file containing metric's metadata (e.g. description, required preprocessing tasks, and accepted input types). It

also contains download links for any necessary models for the metric's implementation and the requester's email address, if provided. Furthermore, this table will include a column that indicates the current status of the metric—whether it's approved, pending, or rejected. By persisting this information, each metric extension request can be reviewed at any time. Moreover, it lays the foundational framework for potential future improvements in semi-automation or fully automation of the metric extension review process.

3.1.4 Proxy Server

The proxy server is introduced to manage the network traffic. Instead of direct communication between the frontend and the backend, requests and responses pass through the proxy server. The connections are to be encrypted with the Transport Layer Security protocol, ensuring that the communications between the client and server are secure and not tampered with. Incoming requests are routed to appropriate backend instances through load balancing. The least connections strategy, of which a request is forwarded to the server with the fewest active connections, is selected to minimize the possibility of server overload.

In addition, connection and rate limiting directives are configured. In this thesis, the connection limit is set to 5 maximum concurrent connections allowed from a single client Internet Protocol (IP) address. The rate of requests is capped at 5 requests per second for each client IP, with a burst capacity of 5 to accommodate potential spike in the network traffic. These policies are introduced to safeguard the platform's availability by preventing any single user from excessively consuming resources, thereby mitigating the risk of denial-of-service attacks. The parameters set for these limits can be easily adjusted based resources availability and usage needs as WUI Auto Evaluate scales up.

3.2 Activity Diagram

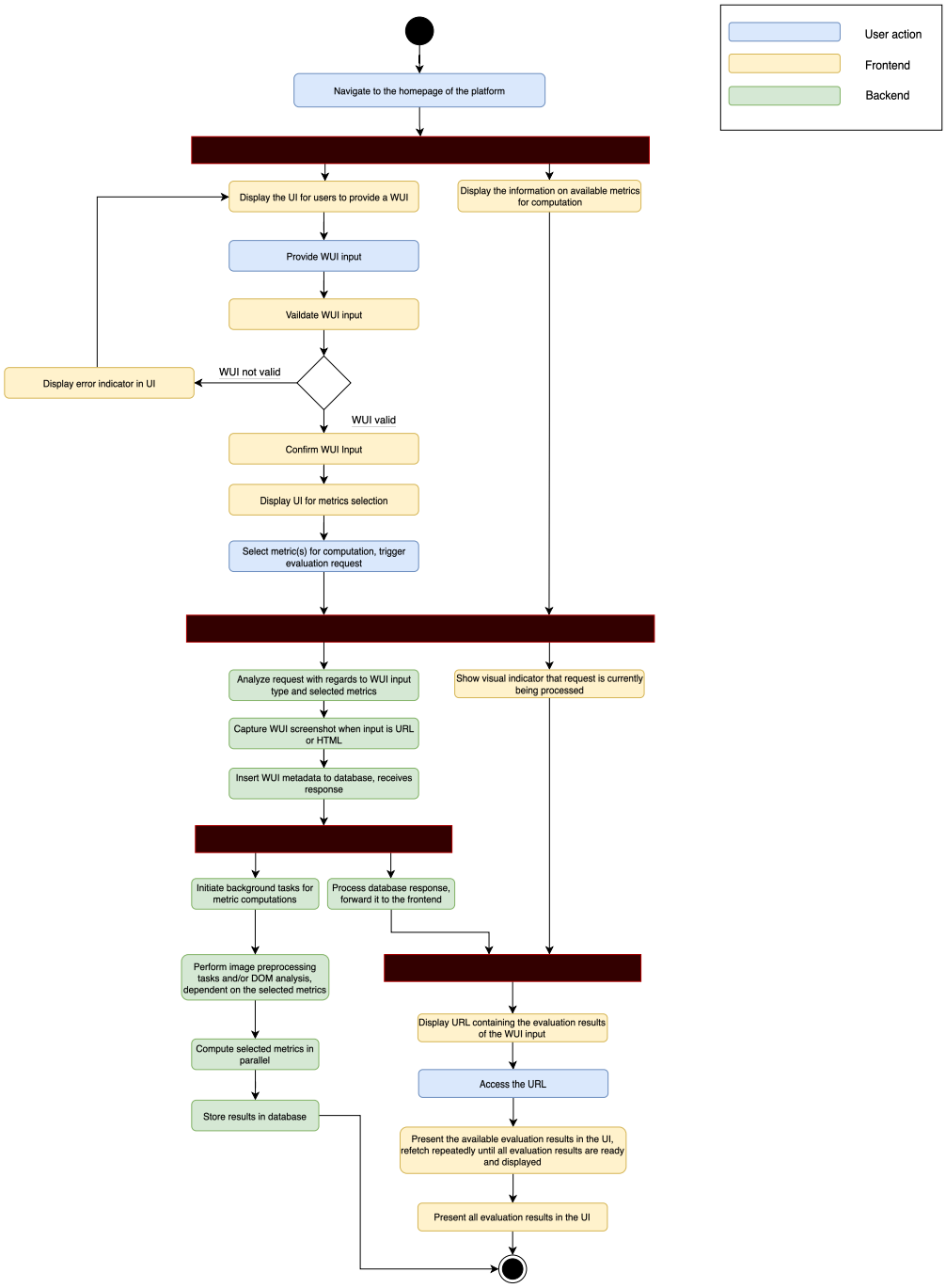


Figure 0.2: Activity Diagram for the WUI Evaluation Process

Figure 3.2 depicts an activity diagram providing a general overview of the workflow for the evaluation of a WUI. For the sake of clarity, some simplifications have been made. For instance, users are able navigate to the homepage at any given phase of the workflow, and this has been omitted in the diagram. In addition, the handling of dependencies is not included in the diagram and considered as part of the metric computation process.

3.3 Summary

This chapter presents the concept for the proposed solution, designed to satisfy the requirements specified in the preceding chapter and ultimately solve the research problems identified in this thesis. The software architecture was illustrated and described in detail, including the reasoning for the selected approaches as well as considerations to alternative options for aspects of the concept. The next chapter will delve on the implementation for realization of the conceptual framework established in this chapter.

4 Implementation

This chapter provides a detailed overview of the technical decisions, approaches, and methodologies employed to realize WUI Auto Evaluate, the solution proposed in the previous chapter. These include the selection of technologies, selection of suitable initial metrics for implementation, aspects of the development of the software architecture components, and rationale behind the decisions taken on the development process.

4.1 Platform and Core Technologies

The development was carried out on a machine running the macOS (version 14) environment with an AArch64[16] processor. However, to resolve compatibility issues and efficiently manage the dependencies of the server-side code, Conda[17] is utilized as an environment manager, and it is configured to emulate an intel-based x86-64[18] architecture.

Python (version 3.9) is selected as the programming language used for the development of the backend, primarily because the majority of metrics and computational models were already implemented or can be easily implemented in Python. In addition, its simple syntax and vast ecosystem of libraries and frameworks simplify the development of functionalities and features required by the solution.

The frontend is written in TypeScript[19], a superset of JavaScript with static type checking, in conjunction with React[20]. This combination is selected to minimize type-associated runtime errors and to enhance the maintainability and readability of the codebase through React's component-based architecture.

The relational database PostgreSQL[21] is used to manage and store the solution's data persistently. It is preferred over NoSQL databases as the data handled within the solution is well-structured and relational (refer to section 3.1.3). In addition, Amazon Simple

[16] https://devblogs.microsoft.com/oldnewthing/20220726-00/?p=106898 [Retrieved: Mar 31, 2024]
[17] https://docs.conda.io/en/latest/ [Retrieved: Mar 31, 2024]
[18] https://web.stanford.edu/class/cs107/guide/x86-64.html [Retrieved: Mar 31, 2024]
[19] https://www.typescriptlang.org [Retrieved: Apr 2, 2024]
[20] https://react.dev/ [Retrieved: Apr 2, 2024]
[21] https://www.postgresql.org [Retrieved: Apr 2, 2024]

Storage Service (S3)[22] is utilized for an efficient and scalable file storage solution. Both the database and file storage are provided by Supabase[23].

Finally, the proxy server is implemented with Nginx[24]. Overall, WUI Auto Evaluate is built with modern, open-source or free, and widely adopted technologies to ensure a highly maintainable and performant system.

Given the size of the project and the number of dependencies (modules & packages) used, only the most pertinent ones will be described in subsequent sections, specifically when they are utilized to address particular functionalities or challenges. For those interested in exploring further, the source code of the platform is available in a public Gitlab repository at the following link: https://gitlab.hrz.tu-chemnitz.de/lcal--tu-chemnitz.de/automated-wui-evaluation.

4.2 Metrics Selection and Implementation

14 metrics have been selected and implemented for WUI Auto Evaluate. These metrics were chosen to encompass a broad spectrum of evaluation criteria, including the evaluation methodologies employed, ranging from simple static code analysis to machine learning-based methods; the nature of the output produced, such as textual data, JSON file, or graphical representation; as well as the complexities of the metric themselves, i.e. from those that are quick and straightforward to compute to those that are computationally expensive. In the following subsection, they are detailed with respect to their names and descriptions, references, required preprocessing steps, and produced output(s).

4.2.1 Metrics Overview

Table 4.1 provides an overview of the implemented metrics along with their assigned IDs and references. Notably, many metric implementations are adapted from the Python-based implementations by AIM [14]. Several adjustments have been made to these implementations to ensure they conform to the interface and structural requirements of WUI Auto Evaluate.

[22] https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html [Retrieved: Apr 2, 2024]
[23] https://supabase.com/docs [Retrieved: Apr 2, 2024]
[24] https://www.nginx.com/ [Retrieved: Apr 8, 2024]

ID	Metric Name	References
m1	PNG file size	[34], [43]
m2	JPEG file size and compression ratio	[34], [43]
m3	Colorfulness	[52]
m4	CIELab color average & standard deviation	[52]
m5	White space proportion	[14], [34]
m6	UIED segmentation	[14], [35]
m7	UMSI (Unified Model of Saliency and Importance)	[39]
m8	Word count	[13]
m9	Edge density	[47], [79]
m10	Feature congestion	[14], [47]
m11	Subband entropy	[14], [47]
m12	Shannon's information entropy	[43]
m13	Accessibility checks	[81]
m14	NIMA (Neural IMage Assessment)	[14], [80]

Table 4.1: ID, Name and References of the Implemented Metrics

m1 – PNG file size

The file size of an image in bytes, saved in the PNG format with 24 bits per pixel in RGB color model. The produced output is a numerical value representing the PNG file size in bytes. Its significance has been empirically validated by numerous research works. For example, studies by Miniukovich & De Angeli [34] and Boychuk & Bakaev [43] have demonstrated its correlation with various aspects of WUI such as visual complexity and aesthetics.

m2 – JPEG file size and compression ratio

This metric analyzes the WUI screenshot converted to the JPEG format with 80% compression quality, calculating the JPEG file size in bytes (24 bits/pixel, RGB) and the compression ratio, which is the size of the original screenshot in PNG format divided by the size of image in JPEG. A conversion to JPEG is the required preprocessing step. This metric produces numerical values for both the file size and compression ratio. Similar to m1, studies in [34] and [43] have demonstrated the correlation between JPEG file size and aspects of WUI such as visual complexity and aesthetics.

m3 – Colorfulness
Quantification of colorfulness in natural images based on a method proposed by Hasler & Süsstrunk [52], which ignores hue. The output produced is a numerical value representing the colorfulness, along with interpretation of the value based on the range described in [52].

m4 – CIELAB color average & standard deviation
The statistical analysis of the color attributes of the WUI screenshot in the CIELAB color space. This metric requires conversion to CIELAB color space preprocessing step. The outputs include the mean and standard deviation for each of the CIELAB color channels, namely L* (perceptual lightness), a* (red-green), and b* (blue-yellow). These statistical measures are utilized by Hasler & Süsstrunk [52] to calculate their colorfulness metric.

m5 – White space proportion
The proportion of white space in the WUI screenshot, which correlates to the quality of the interface's layout [34]. This metric requires image segmentation preprocessing step, which uses the UIED method by Xie et al. [35]. The output produced is a numerical value in the range of 0 to 1, of which a higher value is indicative of poorly distributed content within the WUI. The implementation of this metric is adapted from AIM's implementation [14] for the same metric.

m6 – UIED segmentation
This metric detects and segments the elements within the WUI screenshot with the UIED method [35]. The outputs produced are the UIED-segmented image and a JSON file containing the UIED segmentation results. An example of the JSON output can be found in the Gitlab[25] repository of the project. The implementation of this metric is adapted from AIM's implementation [14] for the same metric.

m7 – UMSI (Unified Model of Saliency and Importance)
This metric predicts the visual importance and saliency map of the input with a deep learning-based model introduced by Fosco et al. [39]. The outputs produced are the heatmap of the importance value and the original WUI screenshot overlaid with the heatmap.

[25] https://gitlab.hrz.tu-chemnitz.de/lcal--tu-chemnitz.de/automated-wui-evaluation/-/blob/main/backend/src/image_preprocessing/segmentation/Readme.md [Retrieved: Apr 4, 2024]

m8 – Word count
Total visible words within the page. It is obtained through static code analysis of the WUI input, relying on the output from the DOM Analyzer component. This metric only accepts WUI input with type URL or HTML. The relevance of this metric has been empirically validated by Ivory et al. [13], which demonstrated that word count, when combined with other page-level metrics, can predict if a website will be highly rated.

m9 – Edge density
The percentage of pixels that are edge pixels, obtained through applying the Canny edge detector[26] to the grayscale image. A higher percentage indicates more visual clutter. Image conversion to the grayscale is required preprocessing step. This metric produces two outputs: a numerical value representing the percentage of pixels and a graphical representation of the edge-detected image. This metric is used by Mack and Oliva [79] as a measure of subjective visual complexity, and its relevance has been further empirically validated by Rosenholtz et al. [47].

m10 – Feature congestion
A measure of display clutter proposed by Rosenholtz et al. [47]. is based upon extensive modeling of the saliency of elements of the WUI input. This metric requires conversion to CIELAB color space preprocessing step. This metric produces two outputs: the numerical feature congestion value and a graphical representation to visualize the congestion. The implementation of this metric is adapted from AIM's implementation [14] for the same metric.

m11 – Subband entropy
Another measure of display clutter introduced by Rosenholtz et al. [47], it is based on measure of the efficiency with which the image can be encoded while maintaining its perceptual image quality. This metric requires conversion to CIELAB color space preprocessing step. The output of the metric is a numerical subband entropy value, of which a higher entropy value indicates increased visual clutter within the image. The implementation of this metric is adapted from AIM's implementation [14] for the same metric.

[26] https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html [Retrieved: Apr 4, 2024]

m12 – Shannon's information entropy

This entropy metric quantifies the amount of information contained within the UI. Higher entropy value indicates a high level of details or noise, implying that the UI has a lot of information. Image conversion to the grayscale is required as the entropy is computed with the grayscale image of the WUI screenshot to ensure that the focus is on the information content rather than color variations. The output of the metric is the numerical entropy value corresponding to the WUI input. The relevance of this metric has been empirically validated in a study by Boychuk & Bakaev [43], which demonstrated that the metric had significant positive correlations with aesthetic and orderliness off the interface.

m13 – Accessibility checks by aXe

Automated analysis of web page content for accessibility testing with axe-core[27], an accessibility testing engine for HTML-based interfaces. This metric only accepts WUI input of type URL. This metric produces two outputs: the number of violations found and a JSON file detailing the accessibility violations found. The open-source Python library developed by Sereduck [81] is employed to compute this metric.

m14 – NIMA (Neural IMage Assessment)

A CNN-based assessment method introduced by Talebi & Milanfar [80] which predicts the distribution of human opinion scores on the technical quality and aesthetics of a given input image. This metric produces two outputs: the NIMA mean score, which ranges from 1 (lowest) to 10 (highest), and the NIMA standard deviation score. The implementation of this metric is adapted from AIM's implementation [14] for the same metric.

[27] https://github.com/dequelabs/axe-core [Retrieved: Apr 5, 2024]

4.2.2 Metric Interface

Each metric is implemented as a Python class, with all classes adhering to a specific Metric interface. This interface is an integral part of WUI Auto Evaluate, as it standardizes the structure and behavior of each metric, which is required to facilitate efficient preprocessing and parallel evaluation of metrics. The definition of the metric interface class is depicted in listing 4.1.

```
class MetricInterface(metaclass=abc.ABCMeta):
    @classmethod
    def __subclasshook__(cls, subclass):
        return (
            hasattr(subclass, "execute")
            and callable(subclass.execute)
            or NotImplemented
        )

    @abc.abstractmethod
    def execute(
        self,
        pil_image: Optional[Image.Image] = None,
        lab_image: Optional[np.ndarray] = None,
        image_url: Optional[HttpUrl] = None,
        grayscale_image: Optional[Image.Image] = None,
        png_image: Optional[BytesIO] = None,
        jpeg_image: Optional[BytesIO] = None,
        segments: Optional[Dict[str, Any]] = None,
        dom_analysis_result: Optional[Dict[str, Any]] = None,
    ) -> Optional[List[Union[int, str, float, Image.Image]]]:
        """
        Execute the metric.

        Returns:
            Results (list of measures)
        Raises:
            NotImplementedError: Implementation is missing.
        """
        raise NotImplementedError
```

Listing 4.1: Metric Interface

MetricInterface is an abstract class that requires all derived subclasses to implement an *execute* method for the metric computation. The *execute* method is designed to be highly versatile, accepting various optional parameters, each corresponding to different representations or aspects of the given WUI input:

- **pil_image**: Pillow[28] image object with PNG format and RGB color model.
- **lab_image**: NumPy[29] array in the CIELAB color space.
- **grayscale_image**: Pillow image object type with grayscale color model.
- **png_image**: Bytes buffer storing the PNG image representation of the input.
- **jpeg_image**: Bytes buffer storing the JPEG image representation of the input.
- **segments**: Dictionary containing the image segmentation (UIED) results.
- **dom_analysis**: Dictionary containing the DOM Analyzer component results.

The *execute* method returns a list comprising integers, strings, floats, or Pillow image objects, accommodating the varied output types produced by implemented metrics.

4.2.3 Metrics Specification

The information of each metric implementation is stored in a JSON file, which follows a structured format to ensure clarity and consistency across the platform. Each metric entry comprises several key-value pairs described as follows:

- **name:** The name of the metric.
- **accepted_input:** An array listing the types of inputs that the metric accepts.
- **description:** The brief description of the metric.
- **preprocessing:** An object specifying preprocessing steps required by the metric. It consists of five fields: **grayscale_conversion_required, jpeg_required, segmentation_required, dom_analysis_required,** and **lab_conversion_required.** Each field signifies whether a particular preprocessing action is necessary, with a Boolean value to indicate its requirement before the metric's computation.
- **references:** An array of objects, each containing **title** and **url** fields, providing citations and relevant literature for the metric.
- **results:** An array of objects describing the expected results. Each object consists of these following fields:

[28] https://pillow.readthedocs.io/en/stable/ [Retrieved: Apr 5, 2024]
[29] https://numpy.org/ [Retrieved: Apr 5, 2024]

- **name:** The name for the metric evaluation result.
- **description:** An optional description for the metric evaluation result. The value for the field should be left empty when a description is not needed.
- **type:** The metric evaluation result type. Three types are supported: "text", "json", and "image". Numerical outputs should have the type "text".
- **scores:** An optional array of objects, each containing **range** and **description** fields, which provide interpretations of the metric evaluation results within specified intervals. This field is optional and should be omitted when such interpretations are not necessary.

Listing A.2 in Appendix A shows an example of a valid metric configuration for the metric with an ID *m12* named Shannon's information entropy.

4.3 Software Components Implementation

This section describes the implementation of the software components outlined in 3.1.

4.3.1 Frontend

The user interface is developed by utilizing Material UI[30], an open-source React component library. It provides ready-to-use building block components with consistent styling, allowing for rapid development of visually appealing and coherent user interfaces. For example, the Autocomplete[31] component is used for the metrics selection interface. Material UI is complemented with Emotion[32], a library for writing Cascading Style Sheets (CSS) in JavaScript. This enables a flexible styling approach to customize Material UI's predefined component styles as necessary.

In addition, FilePond[33] is used for the file uploader component within the Main Interface Page (to accept HTML/PNG WUI inputs) and the Metric Extension Interface (to accept necessary files for metric extension requests). FilePond is chosen as it supports drag and drop functionality, offers preview for image inputs, and has been verified for accessibility[34] as tested with assistive technology software.

[30] https://mui.com/material-ui/ [Retrieved: Apr 5, 2024]
[31] https://mui.com/material-ui/react-autocomplete/ [Retrieved: Apr 8, 2024]
[32] https://emotion.sh/docs/introduction [Retrieved: Apr 5, 2024]
[33] https://pqina.nl/filepond/ [Retrieved: Apr 5, 2024]
[34] https://pqina.nl/filepond/#features [Retrieved: Apr 6, 2024]

For managing network requests, the library ky[35], a modern lightweight HTTP client based on native Fetch API[36], is employed. It provides a promise-based architecture that simplifies the development of asynchronous code. TanStack Query[37] is used in conjunction to handle the server state, caching, and data synchronization. Notably, TanStack Query enables straightforward implementation of the polling mechanism, which is leveraged to facilitate near real-time updates for the evaluation results in the Evaluation Results Dashboard component. Finally, React Context[38] is used to manage the global application state, enabling easy sharing of data across components as required.

Figure 4.1 showcases how the evaluation results presented in WUI Auto Evaluate. For results with graphical value, the output is interactive, where a click would direct users to the URL for viewing the output image at its intrinsic size. For outputs in JSON-type format, they are downloadable by clicking the corresponding download button.

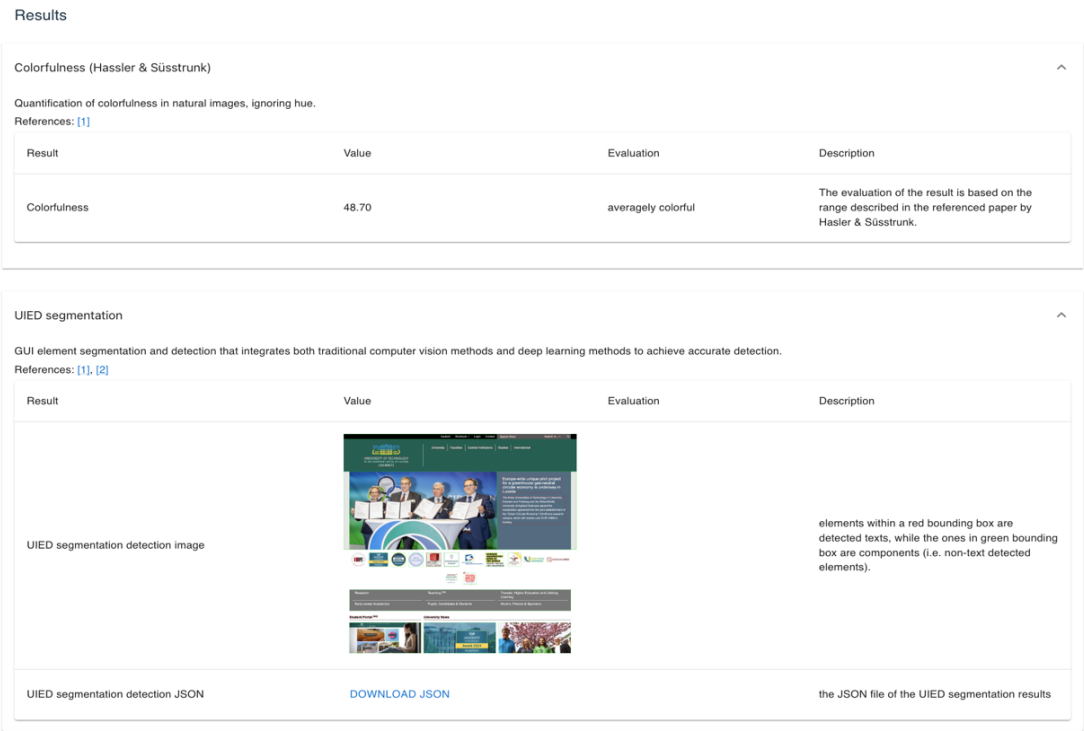


Figure 4.1: Display of metric evaluation results, WUI evaluated: screenshot of TU Chemnitz's homepage (English version).

[35] https://github.com/sindresorhus/ky [Retrieved: Apr 7, 2024]
[36] https://fetch.spec.whatwg.org/#fetch-api [Retrieved: Apr 7, 2024]
[37] https://tanstack.com/query/latest/docs/framework/react/overview [Retrieved: Apr 7, 2024]
[38] https://legacy.reactjs.org/docs/context.html [Retrieved: Apr 7, 2024]

4.3.2 Backend

The Web App Server is built with the FastAPI[39] web framework running under Uvicorn[40], an Asynchronous Server Gateway Interface[41] web server implementation in Python. FastAPI was chosen primarily due to its feature of automatic generation of an OpenAPI specification[42] document based on the source code, providing a clear and standardized documentation for the APIs with comparatively low effort.

For capturing screenshots of WUI inputs, the Selenium WebDriver[43] is utilized. The technology was selected as it is a World Wide Web Consortium (W3C) recommendation[44], indicating that it is a widely adopted standard with robust support across various web environments.

Image processing tasks are mainly handled with the Pillow[45] library. However, both OpenCV[46] and scikit-image[47] are also used for specific tasks which require more complex image manipulation and analysis capabilities not readily available in Pillow. For instance, the UIED segmentation and the computation of feature congestion metric rely on OpenCV, while scikit-image is utilized for image conversion to CIELAB color space, calculation of Shannon's information entropy, and the implementation of Canny edge detector for computing the edge density metric, among others.

The UIED[48] segmentation method, originally written to utilize GoogleOCR[49] or PaddleOCR[50]), has been modified to use Tesseract[51] instead for its Optical Character Recognition (OCR) tasks. This adjustment was motivated by two primary considerations: unlike GoogleOCR which is proprietary and may incur usage fees, Tesseract is open-source and free to use. Furthermore, attempts to integrate PaddleOCR proved to be

[39] https://fastapi.tiangolo.com/ [Retrieved: Apr 7, 2024]
[40] https://www.uvicorn.org [Retrieved: Apr 7, 2024]
[41] https://asgi.readthedocs.io/en/latest/ [Retrieved: Apr 7, 2024]
[42] https://swagger.io/specification/ [Retrieved: Apr 7, 2024]
[43] https://www.selenium.dev/documentation/webdriver [Retrieved: Apr 8, 2024]
[44] https://www.w3.org/TR/webdriver1/ [Retrieved: Apr 8, 2024]
[45] https://pillow.readthedocs.io/en/stable/ [Retrieved: Apr 5, 2024]
[46] https://opencv.org/ [Retrieved: Apr 8, 2024]
[47] https://scikit-image.org [Retrieved: Apr 8, 2024]
[48] https://github.com/MulongXie/UIED [Retrieved: Apr 8, 2024]
[49] https://cloud.google.com/vision/docs/ocr [Retrieved: Apr 8, 2024]
[50] https://github.com/PaddlePaddle/PaddleOCR [Retrieved: Apr 8, 2024]
[51] https://github.com/tesseract-ocr/tesseract [Retrieved: Apr 8, 2024]

impracticable, as the library requires Advanced Vector Extensions instructions [82], which are not supported by the development machine's architecture that uses an AArch64 processor.

Beautiful Soup[52] is used to extract the HTML content of the input required for DOM analysis. Python's standard library importlib[53] is used for dynamically loading or reloading the selected metrics implementations modules, along with their dependencies. A code illustration is provided in listing A.1.

Finally, to facilitate parallel computation of metrics through process-based parallelism, the module pathos[54] is employed. Pathos is preferred due to its usage of dill[55] for data serialization and deserialization, enabling the straightforward handling of complex Python objects. In contrast, Python's standard library multiprocessing[56] uses pickle[57] for its data serialization and deserialization, which cannot handle complex objects as well as dill. For instance, Pillow image objects are not directly serializable with pickle. However, with dill, such objects can be serialized and deserialized directly without the need for conversion to a more primitive data structure, thus simplifying the workflow and enhancing the efficiency of parallel computations of metrics.

4.4 Metrics Extension

WUI Auto Evaluate is developed with extensibility in mind, accommodating the addition of new metrics to the platform through the Metric Extension Interface. Within this interface, user could provide the necessary files and information required for the addition of a new metric. Three specific files are required:

- The metric implementation file: a Python class that implements the Metric Interface specified in section 4.2.2. It must be named following the pattern of `{metric_id}_{metric_description}.py`, as the system will only identify metric implementation files that satisfy this naming requirement.

[52] https://www.crummy.com/software/BeautifulSoup/bs4/doc/ [Retrieved: Apr 8, 2024]
[53] https://docs.python.org/3/library/importlib.html [Retrieved: Apr 8, 2024]
[54] https://pathos.readthedocs.io/en/latest/pathos.html [Retrieved: Apr 8, 2024]
[55] https://dill.readthedocs.io/en/latest/ [Retrieved: Apr 8, 2024]
[56] https://docs.python.org/3/library/multiprocessing.html [Retrieved: Apr 8, 2024]
[57] https://docs.python.org/3/library/pickle.html [Retrieved: Apr 8, 2024]

- A JSON file named `metric.json` that details the proposed metric. The structure of the JSON file must adhere to the specifications outlined in section 4.2.3. Listing A.2 depicts an example of a valid JSON structure for the metric configuration.
- A file listing the modules and packages needed for the metric implementation to run correctly, named `requirements.txt`. The dependency file must be generated with pip[58] or Conda[59].

Furthermore, if the proposed metric requires computational model(s) for its computation, the requisite download links for these models must be supplied via the designated text field element available in the interface. In instances where multiple links are required, each download link must be separated with a comma. Figure 4.2 illustrates the Metric Extension Interface in WUI Auto Evaluate, where users can provide the necessary files and information for requesting a metric extension.

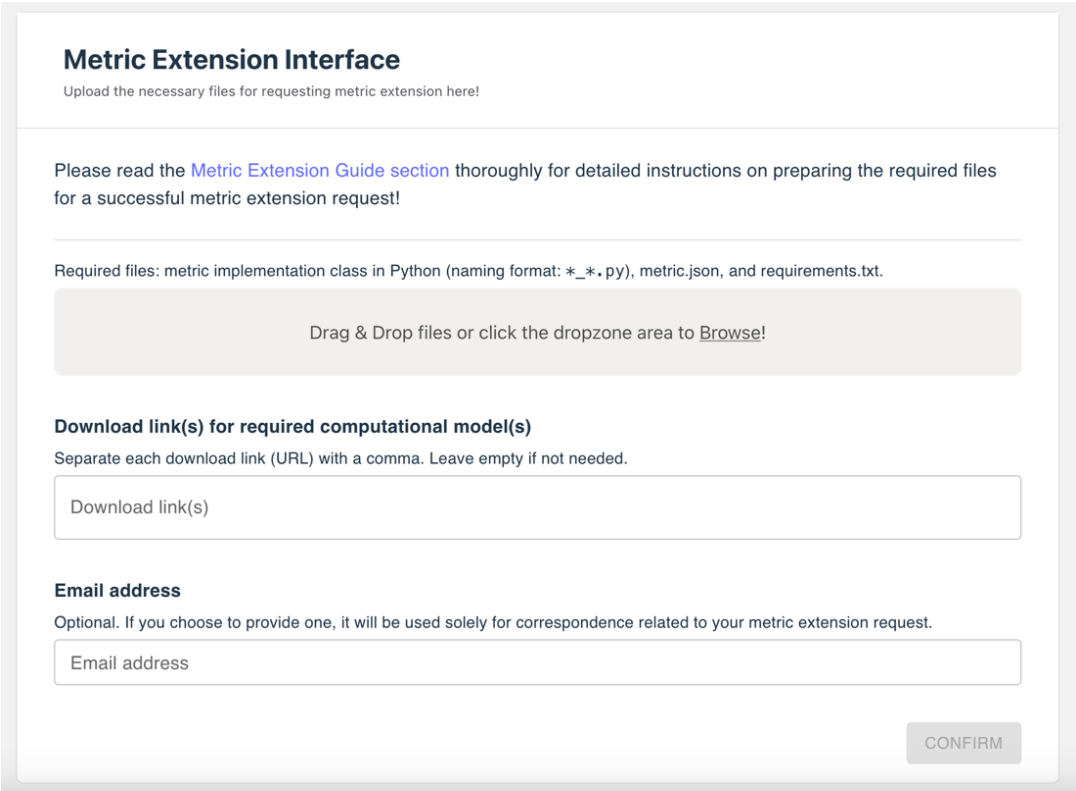


Figure 4.2: Metric Extension Interface of WUI Auto Evaluate

[58] https://pip.pypa.io/en/stable/installation/ [Retrieved: Apr 9, 2024]
[59] https://docs.conda.io/en/latest/ [Retrieved: Mar 31, 2024]

4.5 Summary

This chapter presented the implementation details for the proposed solution, WUI Auto Evaluate, based on the conceptual framework outlined in chapter 3. It covers the selection of technologies, implementation of metrics, technical aspects of the software architecture components, and the process for addition of new metric. Moreover, the rationale behind the development decisions is presented, offering an insight of the strategic considerations guiding the development process to realize the platform.

5 Evaluation

This chapter evaluates the proposed solution WUI Auto Evaluate detailed in the previous chapter. The evaluations encompass two experimental evaluations, which are aimed at assessing the usability and system performance of the proposed solution, and a methodological evaluation according to the requirements established in section 2.1, which includes a comparison to the state-of-the-art approaches described in section 2.2, highlighting the achievements and limitations of the proposed solution.

5.1 Experimental Evaluations

The experimental evaluations for WUI Auto Evaluate consist of two parts: one to measure the user experience, interface intuitiveness, and overall usability of the system, and the other to assess the system performance, focusing on the time performance of metric computations under differing scenarios.

5.1.1 Usability Evaluation

This section describes the experiment evaluation conducted to assess the overall usability, user experience, and attractiveness of WU Auto Evaluate.

Setup. During the experiment, the frontend was hosted on Firebase[60] and accessible via the link https://wui-eval.web.app/, while the backend was run on a MacBook Pro with 32 GB RAM, operating system macOS Sonoma 14.4 and an Aarch64 processor. Prior to engaging with WUI Auto Evaluate, test subjects were required to fill out an initial questionnaire shown in Appendix B.1, which is aimed to collect essential background information of each participant, allowing for a more nuanced analysis of how different demographic groups interacted with and perceived the platform.

Procedure. A virtual meeting session was organized for each test subject. At the start of the experiment, brief background information about WUI Auto Evaluate and an explanation of the experiment's procedure and objectives were provided. Then, the test

[60] https://firebase.google.com/docs/hosting [Retrieved: Apr 9, 2024]

subject was given a task list that they needed to complete using the platform. The task list was designed to acquaint the test subject with the platform's user interface, features, and functionalities. It encompasses evaluating a single WUI, evaluating multiple WUIs with different types in parallel, requesting a metric extension, and an optional task focusing on keyboard navigation for those familiar with this area. The instructions for each task are provided in Appendix B.2.

To ensure consistency across the experiment environment, test subjects were requested to access WUI Auto Evaluate using a desktop-based device and the Google Chrome browser. Furthermore, test subjects were encouraged to verbalize their thought process and ask questions as needed while performing the tasks. After completing the task list, test subjects were encouraged to provide feedback regarding the platform and requested to complete three surveys designed to assess various dimensions of the platform's usability, aesthetics, and user experience. These surveys include System Usability Scale (SUS) by Brooke [83], User Experience Questionnaire (UEQ) by Laugwitz et al. [84], and Attrakdiff by Hassenzahl et al. [85].

SUS. SUS is a simple ten-item questionnaire designed to measure the overall usability of a product. Each item is a 5-level Likert scale ranging from strongly disagree to strongly agree. SUS is technology agnostic, making it applicable across various types of systems and interfaces. The SUS score is a single number representing the overall usability of the product being evaluated, and the score of each individual item is not meaningful on its own [83]. The score is calculated by summing the scores of each item, which range from 0 to 4. For positively worded items, the score contribution is the scale position minus 1. For negatively worded items, the score contribution is 5 minus the scale position [83]. Finally, the sum is multiplied by 2.5, transforming the score to range from 0 to 100.

The validity of SUS has been experimentally evaluated by Bangor et al. [86]. Their findings indicate that the SUS score can and does provide a very useful metric for assessing the overall product's usability. They also found that products that are at least passable have SUS scores above 70, with better-than-average products scoring in the high 70s to upper 80s. Truly superior products score higher than 90, and a SUS score of 50 or below suggests a serious usability failure for the evaluated product.

UEQ. UEQ is a questionnaire designed to measure the user experience of interactive products. It comprises of 26 7-level Likert items ranging from -3 to 3, encompassing six dimensions: Attractiveness, Perspicuity, Efficiency, Dependability, Stimulation, and Novelty. These dimensions capture the ergonomic quality (Perspicuity, Efficiency, and

Dependability), hedonic quality (Stimulation and Originality), and the perceived attractiveness of the product. Ergonomic quality focuses on the goal oriented or task oriented aspects of the product design, i.e. how effective and efficient the product is to use, while hedonic quality focuses on the non-task oriented aspects, such as the originality of the design or the beauty of the user interface [84]. The validation studies conducted in [84] by Laugwitz et al. indicate that the UEQ has a satisfactory level of reliability and construct validity. The questionnaire is available at https://www.ueq-online.org/.

AttrakDiff. AttrakDiff is a questionnaire developed by Hassenzahl et al. [85] designed to measure the perceived pragmatic quality, hedonic quality, and the attractiveness of interactive product designs. Pragmatic quality is concerned to the utilitarian aspect of the product, while hedonic quality is divided into two aspects: hedonic quality – stimulation, focusing on the product's ability to evoke positive emotions and engage the user; and hedonic quality – identity, which focuses on how the product enables users to express their identity through interaction with the product [85]. The questionnaire consists of 28 items, where each item is a 7-level Likert scale ranging from 1 to 7 with contrasting word pairs. A web-based tool named eSurvey[61] is used to administer the questionnaire.

Experimental Data and Results. The experiment was conducted with 13 test subjects (10 males, 3 females) from varied age groups: five test subjects were between 18 to 24 years old, six were between 25 to 34 years old, one was between 35 to 44 years old, and one was 45 years old or older. In terms of linguistic background, the diversity among test subjects were notable: five test subjects spoke mainly German at home, three spoke mainly English, two spoke mainly Hindi, and one each mainly spoke Arabic, Chinese, and Korean. All test subjects have programming experience, although highly varied, ranging from 0.5 to 28 years (mean 6.21, standard deviation 7.30). When excluding the outlier with 28 years of experience, the mean is 4.21, with standard deviation of 2.49.

On a 5-point Likert scale, participants' self-assessed familiarity with web user interfaces yielded a mean score of 2.92, whereas their familiarity with user interface analysis recorded a lower mean score of 2.46. Furthermore, 5 test subjects self-identified to be knowledgeable about keyboard navigation (score 4 or 5), and they were subsequently requested to complete an optional task related this aspect. The complete responses for the initial questionnaire can be seen in Appendix B.3.

[61] https://esurvey.uid.com/project [Retrieved: Apr 14, 2024]

The mean and standard deviation of each task's completion time is shown in table 5.1. The observed high variability in task completion times may be attributed to differing approaches taken by test subjects; some appeared to prioritize completing the tasks as quickly as possible, while others took a more exploratory approach to investigate the platform more in depth. The task completion times of all test subjects can be seen in Appendix B.4.

Task	Mean	Standard Deviation
Task 1	3m 30s	1m 2s
Task 2	2m 24s	59s
Task 3	1m 50s	1m 17s
Task 4	2m 53s	56s

Table 5.1: Mean and standard deviation of task completion times

The mean SUS score of WUI Auto Evaluate from all test subjects was 72.5, with a standard deviation of 14.68. This score suggests that WUI Auto Evaluate has passable usability. However, the relatively high standard deviation suggests that a subset of users experienced difficulties. Interestingly, it is observed that there is a significant variation in mean SUS scores when analyzed by age group. For test subjects aged between 25 to 44 years old (7 test subjects), the mean SUS score was 80.71, indicative of a good perceived usability. However, with test subjects younger than 25 and older than 44 (6 test subjects), the mean SUS score was only 62.91, and notably, it was only individuals from this age range that have rated the platform with a SUS score lower than 70.

This discrepancy suggests that the platform may not be as intuitive to users outside of the central age range. A test subject from the age demographic with the lower mean SUS score provided feedback that might explain this variance: They observed that the platform contains excessively descriptive language and information, leading to confusion. This insight suggests that the platform's usability could potentially be improved by simplifying the language and reducing information density to cater to a broader age range more efficiently. The individual SUS questionnaire responses from each test subject can be found in Appendix B.5.

The evaluation data from UEQ indicates that the platform is perceived to have an average attractiveness (mean of 1.154), above average to excellent pragmatic quality (mean of 1.76), and below average hedonic quality (mean of 0.62). Table 5.2 provides statistical overview of each UEQ scale and figure 5.1 shows comparison of these results to a benchmark data set. The analysis of the UEQ evaluation data was performed with the data analysis tools[62] provided by the authors of the questionnaire. The raw evaluation data and the results for each UEQ item can be seen in Appendix B.6.

Confidence intervals (p=0.05) per scale						
Scale	**Mean**	**Std. Dev.**	**N**	**Confidence**	**Confidence interval**	
Attractiveness	1,154	0,912	13	0,495	0,658	1,649
Perspicuity	1,462	1,079	13	0,587	0,875	2,048
Efficiency	2,019	1,097	13	0,596	1,423	2,615
Dependability	1,788	0,683	13	0,372	1,417	2,160
Stimulation	0,981	1,008	13	0,548	0,433	1,528
Novelty	0,250	1,250	13	0,679	-0,429	0,929

Table 5.2: Statistical overview of UEQ scales

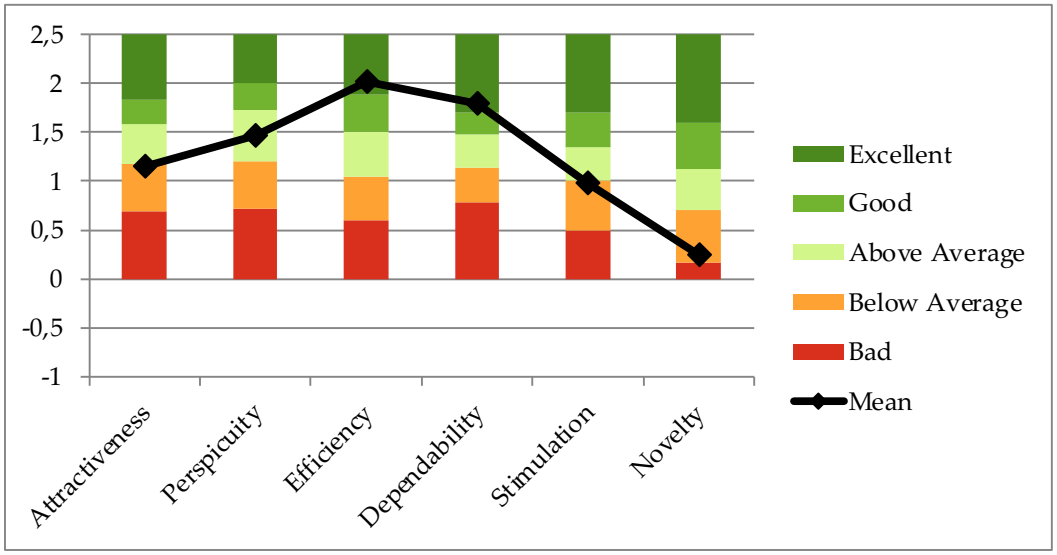


Figure 5.1: Aggregated UEQ Evaluation Data

[62] https://www.ueq-online.org/Material/UEQ_Data_Analysis_Tool.xlsx [Retrieved: Apr 10, 2024]

The evaluation data from AttrakDiff correlated sufficiently well and support the evaluation data from UEQ. The mean values of the pragmatic quality, hedonic quality, and attractiveness were 1.26, 0.4 (mean of hedonic quality – identity: 0.52, mean of hedonic quality – stimulation: 0.29), and 0.68 respectively. Figure 5.2 shows the mean values of each item of the questionnaire. The figure was provided by the survey tool[63] used for this questionnaire. The responses of each test subject for AttrakDiff questionnaire is shown in table B.7.

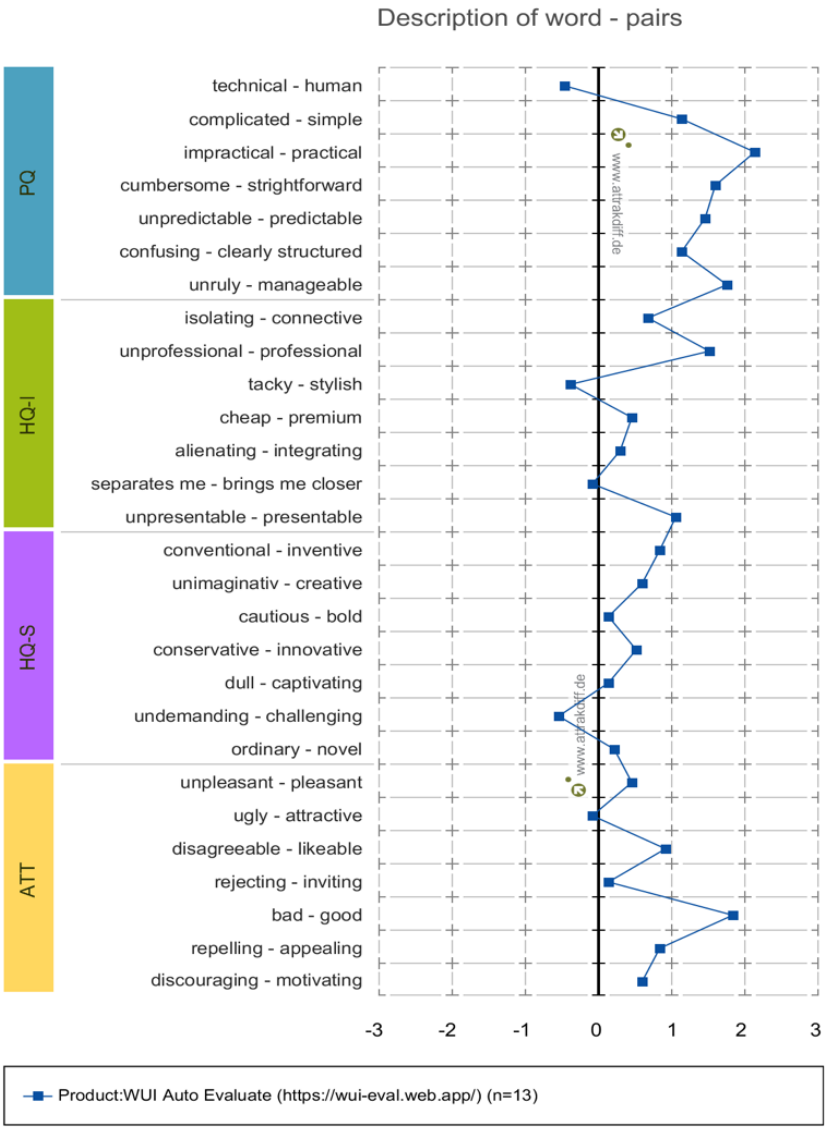


Figure 5.2: Mean values of the word pairs of AttrakDiff questionnaire

[63] https://esurvey.uid.com/ [Retrieved: Apr 10, 2024]

Conclusion. The mean SUS scores reveal that WUI Auto Evaluate has acceptable overall usability, especially for users with an age range of 25 to 44 years old. Evaluation data from both UEQ and AttrakDiff show WUI Auto Evaluate to exhibit notably high pragmatic quality, suggesting that the platform is capable of efficiently meeting the practical needs of its users. On the other hand, the evaluation data also indicate that the platform has an average attractiveness and potentially below-average hedonic quality.

These findings, while highlighting an area of improvement for WUI Auto Evaluate, are somewhat expected given that the requirements defined in section 2.1 were primarily focused on functionalities and features that the platform needs to possess rather than on the user interface design. One requirement that concerns the interface design is focused on accessibility, which was addressed as five test subjects completed the optional task for testing support of keyboard navigation within the platform.

However, given the relatively low number of participants, the generalizability of the experiment results might be limited. In addition, the test subjects had low expertise in user interface analysis (mean of self-assessment: 2.47), suggesting that they might not be the primary target audience of the platform. Nevertheless, the diversity of the test subjects in terms of age groups and main languages spoken was relatively well represented, indicating a broad demographic coverage.

Overall, the experiment provides valuable preliminary insights into the platform's usability and user experience, which seemed to be at an acceptable level. However, to increase the generalizability and reliability of the experiment, more participants with greater expertise in user interface analysis would be required for a more targeted understanding of the platform's usability and user experience.

5.1.2 System Performance Evaluation

This section describes the experiment evaluation conducted to assess the system performance of WUI Auto Evaluate, focusing on the effects of parallelism on the metric computation. The evaluation data and scripts used to perform this experiment can be found in the Gitlab[64] repository within the folder named "system_performance_tests".

Setup. The backend was run on a MacBook Pro with 32 GB RAM, operating system macOS Sonoma 14.4 and an Aarch64 processor**.** cURL[65] was used to measure the duration required for requests to complete. Several adjustments were made to the codebase during the experiment to ensure accurate assessment of metric computation time. These adjustments included the removal of the 5-seconds waiting time in the Screenshot Capturer, the bypassing of the process to save metric evaluation results into the database, and the modification of metric computations to be awaited rather than executed as background tasks. Notably, the process of capturing the screenshot of the input and performing any required preprocessing steps were included in the metric computation time.

Scenarios. Two scenarios were designed:

- **Scenario 1**: Comparison of sequential with parallel metrics computation for fast to compute metrics for a single WUI input.
- **Scenario 2**: Comparison of sequential with parallel metrics computation for moderate to slow to compute metrics for a single WUI input.

Procedure. Before starting the scenario tests, preparatory work was conducted to determine the computational speed of each metric. This was achieved by sending out requests for each metric to the backend server via cURL, using the WUI input of https://wui-eval.web.app/, which is the homepage of the platform itself. Each metric request was repeated 500 times, with exception for the two slowest metrics, feature congestion and subband entropy, of which the request was repeated only 100 times due to limited time. Table 5.3 shows the mean and standard deviation of the computation time for the metrics, sorted by the quickest to the slowest to compute. Figure 5.3 depicts the metric computation times with a box plot, excluding feature congestion and subband entropy metrics, as due to their significantly slower computation times, they would skew the overall scale of the plot. The equivalent plot for these metrics can be respectively found in figure B.8.1 and figure B.8.2 of Appendix B.

[64] https://gitlab.hrz.tu-chemnitz.de/lcal--tu-chemnitz.de/ [Retrieved: Apr 11, 2024]
[65] https://curl.se [Retrieved: Apr 11, 2024]

ID	Metrics	Mean	Standard Deviation
m1	PNG file size	2.41 s	0.168
m3	Colorfulness	2.43 s	0.185
m12	Shannon's information entropy	2.44 s	0.118
m2	JPEG file size and compression ratio	2.46 s	0.194
m9	Edge density	2.66 s	0.128
m14	NIMA	2.67 s	0.123
m4	CIELab color average & standard deviation	3.05 s	0.358
m5	White space proportion	5.42 s	0.224
m6	UIED segmentation	5.38 s	0.147
m8	Word count	8.50 s	0.281
m13	Accessibility checks by aXe	8.89 s	0.328
m7	UMSI	11.55 s	0.406
m11	Subband entropy	42.34 s	1.175
m10	Feature congestion	92.54 s	0.781

Table 5.3: Mean and standard deviation of metrics computation time

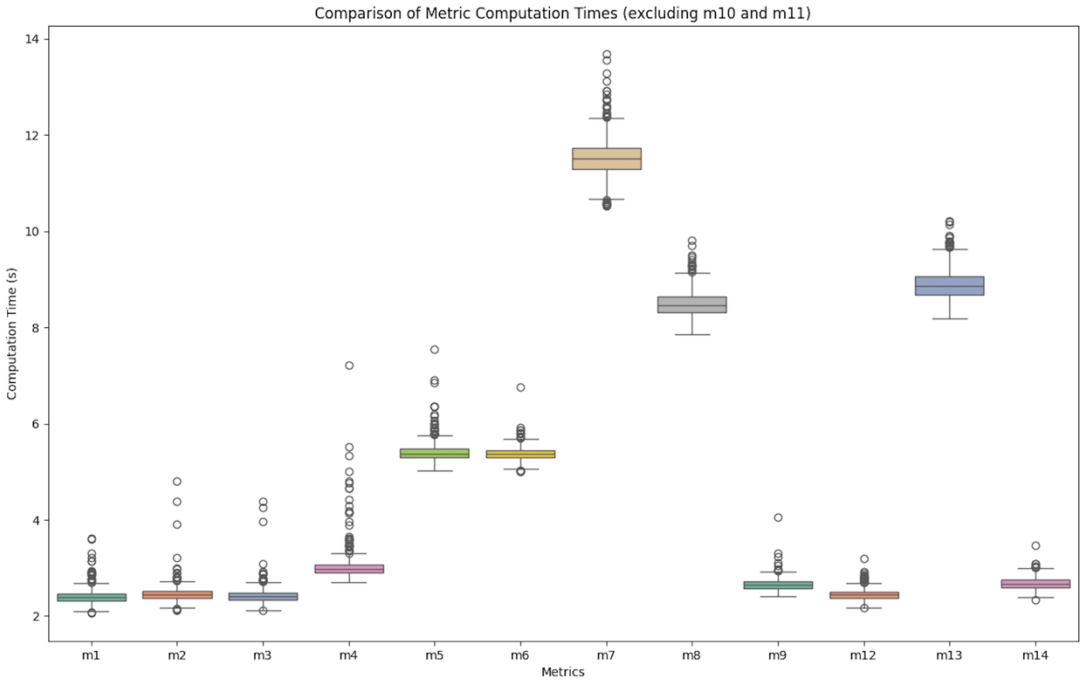


Figure 5.3: Metric computation times (excluding m10 and m11)

Based on the results of the preparatory work, metrics with a mean computation time of under 5 seconds were categorized as fast to compute, while those with mean computation times exceeding 5 seconds were categorized as moderate or slow to compute. With these categories established, Scenario 1 and Scenario 2 were accordingly tested.

Scenario 1. For each request, the metrics computed were PNG file size, JPEG file size and compression ratio, colorfulness, CIELAB color average & standard deviation, edge density, Shannon's information entropy, and NIMA. The WUI input for these requests was a URL: https://wui-eval.web.app/. To ensure the computation time's accuracy and to mitigate the impact of unrelated issues, the requests were conducted 500 times for both sequential and parallel computations. Table 5.4 shows the mean and standard deviation of the computation time for the metrics, revealing that sequential computation was, interestingly, faster than parallel computation in this scenario.

Computation Method	Mean	Standard Deviation
Sequential	3.66 s	0.196
Parallel	4.38 s	0.197

Table 5.4: Mean and standard deviation of metrics computation time for Scenario 1

This could be attributed to the fact that for fast-to-compute metrics, the most time-intensive process was capturing the screenshot of the WUI. Since this capture is performed only once and then shared across all metric computations, the overhead of creating pools for facilitating process-based parallelism might exceed the time saved through parallel processing, thus negating the benefits of parallelism in scenarios where metrics are inherently fast to compute.

Scenario 2. For each request, the metrics computed were white space proportion, UIED segmentation, UMSI, word count, feature congestion, subband entropy, and accessibility checks. The WUI input for these requests was a URL: https://wui-eval.web.app/. Given the time-intensive nature of these metrics and limited time, the requests were conducted 100 times for both sequential and parallel computations. However, as the variability were very low, with standard deviation below 1 for both set of computations, this reduced number of repetitions should not have compromised the validity of this scenario's results. Table 5.5 shows the mean and standard deviation of the computation time for the metrics.

Computation Method	Mean	Standard Deviation
Sequential	151.91 s	0.703
Parallel	97.48 s	0.894

Table 5.5: Mean and standard deviation of metrics computation time for Scenario 2

It can be seen that the parallel computation method was significantly faster than the sequential method. Notably, the mean computation time of the parallel method was close to the mean computation time of the slowest metric, feature congestion (97.48 seconds vs 92.54 seconds). This indicates that the parallel computation method implemented in the platform is efficient, as it reduces the overall computation time to that of the single most time-intensive metric plus a minor additional time required to initiate parallel computation.

Conclusion. This experiment has demonstrated that process-based parallelism is a viable method to significantly accelerate the metrics computation process. The evaluation data from the conducted scenarios reveal that the parallel metrics computation method implemented in the platform offers substantial time performance benefits when the metrics computed require high computation times. However, for metrics that are inherently fast to compute, the practicality of parallel computation diminishes, as the overhead time required to initiate and manage parallel processes could negate the performance benefits associated with the parallel computation. This finding indicates that implementation of an algorithmic strategy to selectively initiate of parallel computation, based on the computational demands of the selected metrics, could further optimize WUI Auto Evaluate's performance, enhancing both time efficiency and resource utilization.

5.2 Requirements Evaluation

In this section, WUI Auto Evaluate is assessed with regard to the requirements elicited in section 2.1. The same evaluation criteria that were used to evaluate the state of the art are applied to the proposed solution. Table 5.6 shows how WUI Auto Evaluate compares to the approaches assessed in section 2.2.

Approach	F1	F2	F3	F4	F5	F6	NF1	NF2
ViCRAM	◐	○	●	○	○	○	○	○
UIED	○	○	●	○	○	○	○	◐
Other Single-metric Approaches	○	○	●	○	○	○	○	○
QUESTIM	●	○	●	○	○	◐	○	○
WAVE	◐	○	●	○	○	○	○	●
Kobold	○	○	●	●	○	●	◐	○
WUI Measurement Platform	●	●	○	○	○	◐	◐	◐
AIM	●	●	◐	○	○	○	◐	◐
WUI Auto Evaluate	●	●	●	●	●	●	◐	◐

Table 5.6: WUI Auto Evaluate in comparison to state of the art (F1: Diverse WUI Input Types Support, F2: Selectable Metrics, F3: Efficient Input Preprocessing, F4: Parallel Processing of Metrics Computation, F5: Parallel Processing of Evaluation Requests, F6: Persistent Evaluation Results, NF1: Extension of New Metrics/Models, NF2: Accessible User Interface Design).

Notably, WUI Auto Evaluate is the only approach that fully satisfies all of the functional requirements (F1 to F6). On the other hand, full fulfillment for non-functional requirements was not achieved. However, they were addressed as indicated by their partial fulfillments. The following paragraphs briefly describe how and why the proposed solution satisfies or partially satisfies these requirements.

F1 – Diverse WUI Input Types Support. WUI Auto Evaluate satisfies this requirement as it supports URL, PNG, and HTML as its WUI input. In addition, the fulfillment of this requirement was also empirically validated in the conducted usability evaluation described in section 5.1.1, as the task list that test subjects had to complete included a task that specifically tested the fulfillment of this requirement (see Appendix B.2, Task 2).

F2 – Selectable Metrics. WUI Auto Evaluate satisfies this requirement through the implementation of metric selection interface which allows for selection and deselection for all of the available metrics within the platform.

F3 – Efficient Input Preprocessing. WUI Auto Evaluate addresses this requirement through the implementation of Metric Dependency Manager, which ensures that only the required preprocessing tasks are executed. They are executed only once, with the results made available to all metric implementations that require them. The scope of these preprocessing tasks encompasses image segmentation, conversion to grayscale,

conversion to JPEG, conversion to CIELAB color space, and DOM Analysis. Furthermore, the capture of input screenshots is also performed only once per WUI input. With these optimizations in place, the requirement *F3* is considered to be fully satisfied.

F4 – Parallel Processing of Metrics Computation. WUI Auto Evaluate satisfies this requirement through the implementation of Parallel Metrics Evaluator, which facilitates the parallel computation of metrics using process-based parallelism. The effectiveness of the implemented parallel metrics computation method has been empirically validated through an experimental evaluation described in section 5.1.2. This evaluation demonstrates that the implemented method offers substantial time performance improvements for metrics that require high computation times, indicating that the approach is efficient and fulfills its original purpose, i.e. to optimize and ultimately speed up computational tasks that are time-intensive.

F5 – Parallel Processing of Evaluation Requests. WUI Auto Evaluate satisfies this requirement through the implementation of user interface and web server which are capable to accept and process multiple WUI inputs simultaneously. With the asynchronous nature of JavaScript and the usage of Promises[66], each evaluation request is handled concurrently and in non-blocking manner, ensuring that an error in one evaluation request does not affect the processing of other requests.

F6 – Persistent Evaluation Results. WUI Auto Evaluate satisfies this requirement through the use of a database to store the metric evaluation results as specified in section 3.1.3, and the implementation of the Evaluation Results Dashboard, which facilitates the presentation of the evaluation results of WUI inputs in unique URLs. Given that these URLs are constructed with the corresponding UUIDs generated by the database, they are virtually guaranteed to be unique, as the chance of collision is close enough to zero to be considered negligible [65].

NF1 – Extension of New Metrics/Models. WUI Auto Evaluate addresses this requirement through the implementation of the Metric Extension Interface, which enables users to suggest new metrics to the platform by providing the necessary files and information as specified in section 4.4. The conducted usability evaluation, described in section 5.1.1, has also empirically validated that the interface developed is capable of facilitating metric extension requests, as all test subjects completed the corresponding task (see Appendix B.2, Task 3). However, given that the addition of a new metric requires manual

[66] https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise [Retrieved: Apr 11, 2024]

approval and integration by an administrator, which must directly modify the codebase, the proposed solution is considered to only partially satisfy *NF1*.

NF2 – Accessible User Interface Design. WUI Auto Evaluate only partially satisfies this requirement. Although the user interface supports keyboard navigation — an aspect empirically validated during the usability evaluation, where several test subjects undertook and completed an optional task pertaining this feature —there exist color contrast errors along with number of alerts within the interface as identified by WAVE. This finding highlights that further refinement is required for the interface to meet the accessibility standards specified by *NF2*, particularly regarding its visual presentation and contrast.

Conclusion. The analysis of the requirements fulfillment showed that WUI Auto Evaluate successfully meets all of the functional requirements. This achievement signifies that the solution has successfully addresses gaps within the research field, as none of the assessed approaches achieve the same level of fulfillment. On the other hand, both non-functional requirements are only partially satisfied, highlighting aspects where further improvement would be beneficial.

5.3 Summary

This chapter presented the evaluations conducted to assess WUI Auto Evaluate. These evaluations include two experimental evaluations focused on the usability and system performance of WUI Auto Evaluate. The usability evaluation demonstrated that WUI Auto Evaluate has an acceptable level of usability, while the system performance evaluation revealed that the implemented parallel metrics computation method offers significant time performance improvements for the computation of metrics that require high computation times. In addition, an evaluation with regard to the requirements elicited in section 2.1 was conducted. The analysis demonstrated that WUI Auto Evaluate has successfully addresses gaps within the research field by achieving overall fulfillment-level that are not matched by other assessed approaches.

6 Conclusion and Outlook

To conclude this thesis, this section summarizes the results, insights, and contributions. Furthermore, a critical consideration of the proposed solution is discussed, along with potential future research work that can be explored.

6.1 Thesis Summary

The purpose of this thesis is to facilitate the automated evaluation of web user interfaces through a single unifying web-based platform. Existing research exhibits a significant gap: the lack of a practical, user-friendly platform capable of efficiently leveraging existing web metrics and assessment models for WUI evaluation. To address this gap, research problems are formulated, and requirements for an appropriate solution have been elicited to address these research problems. The analysis of the state of the art revealed that only a limited number of relevant works share a similar focus and objective. Among these, the primary emphasis tends be only on the core functionalities, while functionalities or features to improve user experience and efficiency of the platform are overlooked. These include the capability to process multiple evaluation requests at a same time, the capability to persistently store the evaluation results in a unique and referenceable URL, accessible user interface, parallel metrics computation, and the possibility to add new evaluation metrics and models to the platform without relying on availability of the platform's source code.

A suitable web-based platform, named WUI Auto Evaluate, has been developed as the proposed solution to address the limitations of existing approaches. The empirical evaluations indicated that WUI Auto Evaluate has an acceptable level of usability and is capable of efficiently meeting the practical needs of its users. The assessment of the solution with regard to the elicited requirements demonstrated that WUI Auto Evaluate achieved an overall fulfillment-level that are unmatched by other assessed approaches, suggesting that the work conducted in this thesis has successfully advanced the field of automated WUI evaluation.

6.2 Contributions

The following paragraph outlines the research contributions of the thesis.

This thesis has systematically analyzed the relevant works within the research field of automated WUI evaluation, identifying the shortcomings of existing works and underscoring the areas of improvement to advance the research field.

To address the identified shortcomings, a web-based platform named WUI Auto Evaluate is designed and created, integrating functionalities and features that are not yet addressed or often overlooked in existing solutions. These include:

- **Parallel processing of evaluation requests**. The capability to accept and process multiple WUI inputs simultaneously.
- **Parallel metrics evaluation**. Implementation of a process-based parallelism technique to significantly speed up the metrics computation process.
- **Persistent result storage**. Persistence of evaluation results that can be accessed through unique URLs.

Finally, empirical experimentations were conducted to assess the usability and system performance of the platform, thereby validating the research works and confirming its practical effectiveness.

6.3 Critical Consideration and Future Work

This section details the limitations of the work produced in this thesis, along with corresponding future work that could be conducted to address these limitations. They are described in the following paragraphs.

Lack of load balancing in metric evaluation. Although a proxy server is implemented within the overall solution framework to facilitate load balancing at the request level, WUI Auto Evaluate does not currently implement dynamic load distribution across server nodes during the metric evaluation process. This limitation results in potentially uneven workload distributions among servers, where one server may be underutilized while another is overwhelmed, as the computational demands of incoming requests can vary significantly.

However, while introducing load balancing during the metric evaluation process could theoretically address this issue, it introduces additional challenges related to inter-server dependencies. For illustration, consider a scenario where multiple server nodes require outputs from the same preprocessing task to evaluate different metrics for the same WUI. This situation necessitates a coordinated approach to data sharing and task synchronization across server nodes, which can be complex to achieve. Moreover, such coordination might adversely affect performance, as the additional time required for sharing and synchronizing data across servers could potentially offset the intended time performance benefits of load balancing. These challenges present a significant opportunity for future work, where sophisticated load distribution algorithms and coordination mechanisms could be developed to optimize the usage of computational resources across multiple servers.

Visual appeal of the user interface design. While the experimental usability evaluation indicated that WUI Auto Evaluate has an acceptable overall usability, it is observed that the hedonic quality scored potentially below average. Furthermore, color contrast accessibility errors were also identified in the interface of WUI Auto Evaluate. These shortcomings suggest that the design of WUI Auto Evaluate does not fully cater to all user needs and should be polished further to enhance its usability and appeal across a broader audience.

Manual integration of new metrics. While WUI Auto Evaluate is developed with extensibility in mind, in its current state, it relies on manual processes to approve and integrate new metrics into the system. Automating these processes would significantly streamline the addition of new metrics, reducing the time and labor costs associated with manual integrations. Future work could focus on developing a more robust and modular framework that supports automated workflows for metric extension. For example, this might include the implementation of advanced methodologies to automatically validate the correctness and reliability of the proposed metrics.

6.3.1 Future Work

This section briefly outlines two potential areas for future research opportunities that, although not within the scope of this thesis, should be explored in future work.

Aggregation of metric evaluation results to form a holistic assessment. Future research could work on developing a comprehensive framework that aggregates individual metric evaluations into a holistic assessment of the WUI input. This would require thorough investigations of the metrics to understand their relations and relevance. Multidisciplinary approaches involving user experience research, statistical analysis, and machine learning techniques might be essential to develop the framework that could achieve this research objective. Ultimately, such research could offer a more nuanced understanding of how different aspects of a WUI interact and collectively influence the overall quality.

Automated remediation suggestions based on the UI evaluation. Although numerous metrics and models have been defined and developed to automate the evaluation of user interfaces, the majority of these works, as observed by the author, primarily focus on only the evaluation part, without providing solutions nor suggestions to fix the identified issues or to improve the UI, thereby limiting the usefulness of the evaluation results. One notable exception is the work by Grigera et al. [66], in which they developed a tool that automatically suggests or in some cases refactors the UI based on the detected usability issues. However, they only address usability, and for other aspects of UI, such as visual complexity or aesthetics, there remains a significant opportunity and need for further research and development.

Bibliography

[1] C.-M. Karat (1990). “Cost-Benefit Analysis of Usability Engineering Techniques,” *Proceedings of the Human Factors Society Annual Meeting*, vol. 34, no. 12, pp. 839–843. DOI: 10.1177/154193129003401203

[2] J. Nielsen and V. L. Phillips (1993). “Estimating the relative usability of two interfaces,” *Proceedings of the SIGCHI conference on Human factors in computing systems - CHI ’93*, pp. 214–221. DOI: 10.1145/169059.169173

[3] D. Stone, C. Jarrett, M. Woodroffe, and S. Minocha, “Introduction” in *User Interface Design and Evaluation*, Amsterdam, The Netherlands: Elsevier, 2005, pp. 3–24.

[4] J. Nielsen and R. Molich (1990). “Heuristic evaluation of user interfaces,” *Proceedings of the SIGCHI conference on Human factors in computing systems Empowering people - CHI ’90*, pp. 249–256. DOI: 10.1145/97243.97281

[5] C. Lewis and J. Rieman, “Testing the Design with Users” in *Task-Centered User Interface Design: A Practical Introduction*, Boulder, CO, USA: Dep. of Computer Science, Univ. of Colorado, 1993, pp. 77–96.

[6] P. G. Polson, C. Lewis, J. Rieman, and C. Wharton (1992). “Cognitive walkthroughs: a method for theory-based evaluation of user interfaces,” *International Journal of Man-Machine Studies*, vol. 36, no. 5, pp. 741–773. DOI: 10.1016/0020-7373(92)90039-N

[7] W. Hwang and G. Salvendy (2010). “Number of people required for usability evaluation,” *Communications of the ACM*, vol. 53, no. 5, pp. 130–133. DOI: 10.1145/1735223.1735255

[8] R. Molich et al. (1999). “Comparative evaluation of usability tests,” *CHI ’99 extended abstracts on Human factors in computing systems - CHI ’99*, pp. 83–84. DOI: 10.1145/632716.632768

[9] M. Hertzum and N. E. Jacobsen (2003). “The Evaluator Effect: A Chilling Fact About Usability Evaluation Methods,” *International Journal of Human-Computer Interaction*, vol. 15, no. 1, pp. 183–204. DOI: 10.1207/S15327590IJHC1501_14

[10] M. Y. Ivory and M. A. Hearst (2001). “The state of the art in automating usability evaluation of user interfaces,” *ACM Computing Surveys*, vol. 33, no. 4, pp. 470–516. DOI: 10.1145/503112.503114

[11] A. Dingli and S. Cassar (2014), “An Intelligent Framework for Website Usability,” *Advances in Human-Computer Interaction*, vol. 2014, pp. 1–13. DOI: 10.1155/2014/479286

[12] E. H. Chi et al. (2003). “The bloodhound project,” *Human Factors in Computing Systems*, pp. 505–512. DOI: 10.1145/642611.642699

[13] M. Y. Ivory, R. R. Sinha, and M. A. Hearst (2001). “Empirically validated web page design metrics,” *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pp. 53–60. DOI: 10.1145/365024.365035

[14] A. Oulasvirta et al. (2018). "Aalto Interface Metrics (AIM): A Service and Codebase for Computational GUI Evaluation". *Adjunct Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology (UIST '18 Adjunct),* pp. 16–19. DOI: 10.1145/3266037.3266087

[15] O. Wu, W. Hu, and L. Shi (2013). “Measuring the Visual Complexities of Web Pages”. *ACM Transactions on the Web,* vol. 7, no. 1, pp. 1–34. DOI: 10.1145/2435215.2435216

[16] Q. Dou, X. S. Zheng, T. Sun, P-A. Heng (2019). “Webthetics: Quantifying Webpage Aesthetics with Deep Learning”. *International Journal of Human-Computer Studies,* vol. 124, pp. 56–66. DOI: 10.1016/j.ijhcs.2018.11.006

[17] M. Bakaev, S. Heil, L. Chirkov, and M. Gaedke (2022). “Benchmarking Neural Networks-Based Approaches for Predicting Visual Perception of User Interfaces”. *Artificial Intelligence in* HCI, pp. 217–231. Lecture *Notes in Computer Science*, vol. 13336. Springer, Cham. DOI: 10.1007/978-3-031-05643-7_14

[18] A. Keselj, M. Milicevic, K. Zubrinic, and Z. Car (2022). “The Application of Deep Learning for the Evaluation of User Interfaces”. *Sensors*, vol. 22, no. 23, p. 9336. DOI: 10.3390/s22239336

[19] A. Delitzas, K. C. Chatzidimitriou, and A. L. Symeonidis (2023). "Calista: A Deep-Learning Based System for Understanding and Evaluating Website Aesthetics". *International Journal of Human-Computer Studies,* volume 175, p. 103019. DOI: 10.1016/j.ijhcs.2023.103019

[20] A. Taivalsaari, T. Mikkonen, M. Anttonen, and A. Salminen (2011). “The Death of Binary Software: End User Software Moves to the Web”. *9th International Conference on Creating, Connecting and Collaborating through Computing*, pp. 17–23. DOI: 10.1109/C5.2011.9

[21] A. Taivalsari and T. Mikkonen (2011). “The Web as an Application Platform: The Saga Continues”. *Software Engineering and Advanced Applications (SEAA), 2011 37th EUROMICRO Conference*, pp. 170–174. DOI: 10.1109/SEAA.2011.35

[22] S. Heil (2021). “Web Migration Revisited: Addressing Effort and Risk Concerns”. In: *Doctoral Dissertations in Web Engineering and Web Science*, pp. 1–329.

[23] S. Charfi, T. Abdelwaheb, H. Ezzedine, and C. Kolski (2014). “Widgets Dedicated to User Interface Evaluation”. *International Journals of HCI,* vol. 30, no. 5, pp. 408–421. DOI: 10.1080/10447318.2013.873280

[24] E. Allen (2001). “Designing Extensible Applications” in *Diagnosing Java Code,* IBM developerWorks, 2001.

[25] M. Zenger (2004). “Extensible Component-Based Software” in *Programming Language Abstractions for Extensible Software Component*, Lausanne, Switzerland: EPFL, 2004, pp. 1–17. DOI: 10.5075/epfl-thesis-2930

[26] A. H. Doan et al. (2009). “Information Extraction Challenges in Managing Unstrutured Data”. *ACM SIGMOD Record,* vol. 37, no. 4, pp. 14–20. DOI: 10.1145/1519103.1519106

[27] World Wide Web Consortium (2023)."Web Content Accessibility Guidelines (WCAG) 2.2, Understanding SC 2.1.1: Keyboard (Level A)". URL: https://www.w3.org/WAI/WCAG22/Understanding/keyboard (visited on December 17, 2023)

[28] C. A. Szyperski (1996). "Independently Extensible Systems - Software Engineering Potential and Challenges". *Proceedings 19th Australian Computer Sci Conference.* Available at: https://citeseerx.ist.psu.edu/document?type=pdf&doi=036510ff531b7b1f31197e8c2a5b8f511a666757

[29] IETF (1997). "Key Words for Use in RFCs to Indicate Requirement Levels". Available at: https://www.ietf.org/rfc/rfc2119.txt

[30] E. Michailidou, S. Eraslan, Y. Yesilada, and S. Harper (2021). "Automated Prediction of Visual Complexity of Web Pages: Tools and Evaluations". *International Journals of HCI,* vol. 145, p. 102523. DOI: 10.1016/j.ijhcs.2020.102523

[31] H. C. Purchase, J. Hamer, A. Jamieson, and O. Ryan (2011). "Investigating Objective Measures of Web Page Aesthetics and Usability". *AUIC'11: Proceedings of Twelfth Australasian UI Conference*, vol. 117, pp. 19–28.

[32] M. Bakaev, S. Heil, V. Khvorostov, and M. Gaedke (2018). "HCI Vision for Automated Analysis and Mining of Web User Interfaces". *Proceedings of 18th International Conference on Web Engineering*, pp. 136–144. *Lecture Notes in Computer Science*, vol. 10845. Springer, Cham. DOI: 10.1007/978-3-319-91662-0_10

[33] J. Kong et al. (2012). "Web Interface Interpretation Using Graph Grammars". *IEEE Transaction on Systems, Man, and Cybernetics Part C (Applications and Reviews)*, vol. 42, no. 4, pp. 590–602. DOI: 10.1109/TSMCC.2011.2171335

[34] A. Miniukovich and A. De Angeli (2015). "Computation of Interface Aesthetics". *CHI'15: Proceedings of the 33rd Annual ACM Conference on Human Factor in Computing Systems*, pp. 1163–1172. DOI: 10.1145/2702123.2702575

[35] M. Xie, S. Feng, Z. Xing, J. Chen, and C. Chen (2020). "UIED: A Hybrid Tool for GUI Element Detection". *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 1655–1659. DOI: 10.1145/3368089.3417940

[36] A. Riegler and C. Holzmann (2018). "Measuring Visual User Interface Complexity of Mobile Applications with Metrics". *Interacting with Computers, 30, 3,* pp. 207–223. DOI: 10.1093/iwc/iwy008

[37] J. Abascal, M. Arrue and X. Valencia (2019). "Tools for Web Accessibility Evaluation". In: *Web Accessibility*, pp. 479–503. DOI: 10.1007/978-1-4471-7440-0_26

[38] European Union (2019). "Directive (EU) 2019/882 of the European Parliament and of the Council of 17 April 2019 on the Accessibility Requirements for Products

and Services (Text with EEA Relevance)". URL: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A32019L0882 (visited on January 2, 2024)

[39] C. Fosco, V. Casser, A. K. Bedi, P. O'Donovan, A. Hertzmann, and Z. Bylinskii (2020). "Predicting Visual Importance Across Graphic Design Types" In: *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology*, pp. 249–260. DOI: 10.1145/3379337.3415825

[40] Z. Bylinskii, T. Judd, A. Oliva, A. Torralba, and F. Durand (2019). "What Do Different Evaluation Metrics Tell Us About Saliency Models?" *IEEE Transactions on Pattern Analysis and Machine Intelligence 41, 3*, pp. 740–757. DOI: 10.1109/TPAMI.2018.2815601

[41] K. Reinecke and K. Z. Gajos (2014). "Quantifying Visual Preferences Around the World". *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pp. 11–20. DOI: 10.1145/2556288.2557052

[42] M. G. Khani, M. R. Mazinani, M. Fayyaz and M. Hoseini (2016). "A Novel Approach for Website Aesthetic Evaluation Based on Convolutional Neural Networks". *2016 Second International Conference on Web Research*, pp. 48–53. DOI: 10.1109/ICWR.2016.7498445

[43] E. Boychuk and M. Bakaev (2019). "Entropy and Compression Based Analysis of Web User Interfaces". *International Conference on Web Engineering (ICWE) 2019*, pp. 253–261. *Lecture Notes in Computer Science*, vol. 11496. Springer, Cham. DOI: 10.1007/978-3-030-19274-7_19

[44] Z. Bylinskii et al. (2017). "Learning Visual Importance for Graphic Designs and Data Visualizations". *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology*, pp: 57–69. DOI: 10.1145/3126594.3126653

[45] P. O'Donovan, A. Agarwala, and A. Hertzmann (2014). "Learning Layouts for Single-Page Graphic Designs". *IEEE Transactions on Visualization and Computer Graphics,* vol. 20, no. 8, pp. 1200–1213. DOI: 10.1109/TVCG.2014.48

[46] M. Cornia, L. Baraldi, G. Serra, and R. Cucchiara (2018). "Predicting Human Eye Fixations via an LSTM-based Saliency Attentive Model". In: *IEEE Transactions on Image Processing,* vol. 27, no. 10, pp. 5142–5154. DOI: 10.1109/TIP.2018.2851672

[47] R. Rosenholtz, Y. Li, and L. Nakano (2007). "Measuring Visual Clutter". In: *Journal of Vision August 2007,* vol. 7, no. 2, p. 17. DOI: 10.1167/7.2.17

[48] D. C. L. Ngo and J. G. Byrne (2001). "Another Look at a Model for Evaluating Interface Aesthetics". *International Journal of Applied Mathematics and Computer Science*, vol. 11, no. 2, pp. 515–535. Available at: https://eudml.org/doc/207518

[49] D. C. L. Ngo, L. S. Teo and J. G. Byrne (2003). "Modelling Interface Aesthetics". *Information Sciences,* vol. 152, pp. 25–46. DOI: 10.1016/S0020-0255(02)00404-8

[50] K. Reinecke et al. (2013). "Predicting User' First Impression of Website Aesthetics with Quantification of Perceived Visual Complexity and Colorfulness". *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pp. 2049–2058. DOI: 10.1145/2470654.2481281

[51] M. Zen and J. Vanderdonckt (2013). "Metric-Based Evaluation of Graphical User Interfaces: Model, Method and Software Support ". *Proceedings of the 5th ACM SIGCHI Symposium on Engineering Interactive Computing Systems*, pp. 183–186. DOI: 10.1145/2494603.2480331

[52] D. Hasler and S. Süsstrunk (2003). "Measuring Colorfulness in Natural Images". *Proceedings of IS&T/SPIE Electronic Image 2003: Human Vision and Electronic Imaging VIII*, pp. 87–95. DOI: 10.1117/12.477378

[53] S. N. Yendrikhovskij, F. J. J. Blommaert, and H. de Ridder (1998). "Optimizing Color Reproduction of Natural Images". *Proceedings of Imaging Science and Technology 6th Color and Imaging Conference*, pp. 140–145. DOI: 10.2352/CIC.1998.6.1.art00029

[54] WebAIM at Utah State University (1999). "WAVE (Web Accessibility Evaluation Tool)". URL: https://wave.webaim.org (visited on January 13, 2024)

[55] T. A. Nguyen and C. Csallner (2015). "Reverse Engineering Mobile Application User Interfaces with REMAUI (T)". *2015 30th IEEE/ACM International Conference on Automate Software Engineering (ASE)*, pp. 248–259. DOI: 10.1109/ASE.2015.32

[56] C. Yongxin, Z. Tonghui, and C. Jie (2019). "UI2Code: How to Fine-tune Background and Foreground Analysis". URL: https://medium.com/hackernoon/ui2code-how-to-fine-tune-background-and-foreground-analysis-fb269edcd12c (visited on January 14, 2024)

[57] S. Ren, K. He, R. Girshick and J. Sun (2017). "Faster R-CNN: Towards Real-Time Object Detection with Region Proposed Networks". *IEEE Transactions on Pattern Analysis & Machine Intelligence,* vol. 39, no. 6, pp. 1137–1149. DOI: 10.1109/TPAMI.2016.2577031

[58] J. Redmon and A. Farhadi (2018). "YOLOv3: An Incremental Improvement". In: *arXiv preprint arXiv:1804.02767*. DOI: 10.48550/arXiv.1804.02767

[59] K. Duan, S. Bai, L. Xie, H. Qi, Q. Huang, and Q. Tian (2019). "Centernet: Keypoint Triplets for Object Detection". *2019 IEEE/CVF International Conference on Computer Vision*, pp. 6568–6577. DOI: 10.1109/ICCV.2019.00667

[60] R. Reber, N. Schwarz, and P. Winkielman (2004). "Processing Fluency and Aesthetic Pleasure: Is Beauty in the Perceiver's Processing Experience?".

Personality and Social Psychology Review, vol. 8 (4), pp. 364–382. DOI: 10.1207/s15327957pspr0804_3

[61] M. Bakaev, M. Speicher, S. Heil and M. Gaedke (2020). "I Don't Have That Much Data! Reusing User Behavior Models for Website from Different Domains". *International Conference on Web Engineering (ICWE) 2020*, pp. 146–162. *Lecture Notes in Computer Science,* vol. 12128. Springer, Cham. DOI: 10.1007/978-3-030-50578-3_11

[62] M. Bakaev, S. Heil, V. Khvorostov, and M. Gaedke (2019). "Auto-Extraction and Integration of Metrics for Web User Interfaces". *Journal of Web Engineering 17 (6-7)*, pp. 561-590. ISSN: 1540-9589

[63] M. Bakaev, S. Heil, V. Khvorostov, and M. Gaedke (2017). "Evaluation of User-Subjective Web Interface Similarity with Kansei Engineering-based ANN". 2017 *IEEE 3rd International Workshop on Usability and Accessibility focused Requirements Engineering*, pp. 125–131. DOI: 10.1109/REW.2017.13

[64] M. Nagamachi (1995). "Kansei Engineering: A New Ergonomic Customer-oriented Technology for Product Development". *International Journal of Industrial Ergonomics,* vol. 15, no. 1, pp. 3–11. DOI: 10.1016/0169-8141(94)00052-5

[65] J. Hall (2021). "What Are the Odds?". URL: https://jhall.io/archive/2021/05/19/what-are-the-odds/ (visited on April 11, 2024)

[66] J. Grigera, A. Garrido, and G. Rossi (2017). "Kobold: Web Usability as a Service". *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering,* pp. 990–995. DOI: 10.1109/ASE.2017.8115717

[67] J. Grigera, A. Garrido, J. M. Rivero, and G. Rossi (2017). "Automatic Detection of Usability Smells in Web Applications". *International Journal of Human-Computer Sciences*, pp. 129–148. DOI: 10.1016/j.ijhcs.2016.09.009

[68] M. Zen and J. Vanderdonckt (2014). "Towards an Evaluation of Graphical User Interface Aesthetics based on Metrics". *2014 IEEE Eighth International Conference on Research Challenges in Information Sciences*, pp. 1–12. DOI: 10.1109/RCIS.2014.6861050

[69] M. Zen and J. Vanderdonckt (2016). "Assessing User Interface Aesthetics Based on the Intersubjectivity of Judgment". *Proceedings of the 30th International BCS Human Computer Interaction Conference: Fusion!,* pp. 1–12. DOI: 10.14236/ewic/HCI2016.25

[70] M. Bakaev, S. Heil, N. Perminov, and M. Gaedke (2019). "Integration Platform for Metric-Based Analysis of Web User Interfaces". *Proceedings of 19th International Conference of Web Engineering (ICWE2019)*. DOI: 10.1007/978-3-030-19274-7_39

[71] M. Bakaev, V. Khvorostov, S. Heil, and M. Gaedke (2017). "Web Intelligence Linked Open Data for Website Design Reuse". *Proceedings of 17th International Conference of Web Engineering (ICWE2017)*, pp. 370–377. DOI: 10.1007/978-3-319-60131-1_22

[72] World Wide Web Consortium (1998). "What is the Document Object Model?". URL: https://www.w3.org/TR/WD-DOM/introduction.html (visited on February 27, 2024)

[73] Python Software Foundation (2024). "multiprocessing – Process-based Parallelism". *Python 3.12.2 Documentation*. URL: https://docs.python.org/3.12/library/multiprocessing.html (visited on February 29, 2024)

[74] K.D. Lee and S. Hubbard, "Distributed Multiprocessing" in *Data Structures and Algorithms with Python: With an Introduction to Multiprocessing*, *Springer, Cham, Switzerland*: Springer, 2024, pp. 315-344.

[75] Object Management Group (2017). "OMG Unified Modeling Language Version 2.5.1". URL: https://www.omg.org/spec/UML/2.5.1/PDF (visited on March 19, 2024)

[76] ISO/CIE 11664-4:2019 (2019). "Colorimetry, Part 4: CIE 1976 L*a*b Colour Space". URL: https://www.iso.org/standard/74166.html (visited on March 20, 2024).

[77] C. Ellis and R. Brandl (2023). "Website Loading Time Statistics (2023): Average Page Load Time and Bounce Rate. URL: https://www.tooltester.com/en/blog/website-loading-time-statistics/ (visited on March 21, 2024).

[78] P. J. Leach, R. Salz, and M. H. Mealing (2005). "A Universally Unique IDentifier (UUID) URN Namespace". URL: https://datatracker.ietf.org/doc/html/rfc4122

[79] M. L. Mack and A. Oliva (2004). "Computational estimation of visual complexity". *12th Annual Object, Perception, Attention, and Memory Conference.*

[80] H. Talebi and P. Milanfar (2018). "Neural Image Assessment". *IEEE Transactions on Image Processing, vol. 27, no. 8.* DOI: 10.1109/TIP.2018.2831899

[81] K. Sereduck (2018). "axe-selenium-python 2.1.6". URL: https://pypi.org/project/axe-selenium-python/ (visited on April 3, 2024)

[82] C. Lomont (2011). "Introduction to Intel® Advanced Vector Extensions". URL: https://hpc.llnl.gov/sites/default/files/intelAVXintro.pdf (visited on April 9, 2024)

[83] J. Brooke (1996). "SUS - A Quick and Dirty Usability Scale". *Usability Evaluation in Industry*, pp. 189-194. ISBN: 9780429157011

[84] B. Laugwitz, T. Held and M. Schrepp (2008). "Construction and Evaluation of a User Experience Questionnaire". *4th Symposium of the Workgroup Human-Computer Interaction and Usability Engineering of the Austrian Computer*

Society, USAB 2008: HCI and Usability for Education and Work, pp. 63–76. DOI: 10.1007/978-3-540-89350-9_6

[85] M. Hassenzahl, M. Burmester, and F. Koller (2003). "Attrakdiff: Ein Fragebogen zur Messung wahrgenommener hedonischer und pragmatischer Qualität". *J. Ziegler & G. Szwillus (Eds.), Mensch & Computer 2003*, pp. 187–196. DOI: 10.1007/978-3-322-80058-9_19

[86] A. Bangor, P. T. Kortum, and J. T. Miller (2008). "An Empirical Evaluation of the System Usability Scale". I*nternational Journal of Human-Computer Interaction, volume 24:6,* pp. 574–594. DOI: 10.1080/10447310802205776

Appendix A Implementation Details

```
from pathlib import Path
import importlib
import sys

metric_modules = []
# Locate metrics implementations
for metric in self.metrics_to_evaluate:
    metric_files = [
        metric_file for metric_file
        in list(Path(METRICS_DIR).glob(METRICS_FILE_PATTERN))
        if metric_file.name.startswith(metric + "_")
    ]

    if metric_files:
        module_name = re.sub(r"/|\\", ".", str(metric_files[0].parent))
        # 'stem' gives the filename without extension
        metric_module_name = metric_files[0].stem

        # Construct the full module import path
        full_module_name = f"{module_name}.{metric_module_name}"
        try:
            if full_module_name in sys.modules:
                metric_module = importlib.reload(sys.modules[full_module_name])
            else:
                metric_module = importlib.import_module(full_module_name)
            metric_modules.append(metric_module)
        except ModuleNotFoundError as e:
            print(f"Failed to import {full_module_name}: {e}")
    else:
        print('metric not found!')
```

Listing A.1: Dynamic Import for Metric Implementations

```
{
  "m12": {
    "name": "Shannon's information entropy",
    "description": "A measure of unpredictability and uncertainty of in-
formation in a set of data.",
    "accepted_input": ["url", "html", "png"],
    "preprocessing": {
      "grayscale_conversion_required": true,
      "segmentation_required": false,
      "jpeg_conversion_required": false,
      "dom_analysis_required": false,
      "lab_conversion_required": false
    },
    "references": [
      {
        "title": "E. Boychuk and M. Bakaev (2019). Entropy and Compression
Based Analysis of Web User Interfaces. International Conference on Web En-
gineering (ICWE) 2019, pp. 253-261.",
        "url": "https://doi.org/10.1007/978-3-030-19274-7_19"
      }
    ],
    "results": [
      {
        "name": "Shannon's information entropy",
        "description": "Higher entropy value indicates a higher level of
detail or noise, implying the UI has a lot of information.",
        "type": "text"
      }
    ]
  }
}
```

Listing A.2: Example of a valid metric configuration

Appendix B Evaluation Materials

This appendix contains the materials used in the experimental evaluations performed in this thesis.

B.1 Initial Questionnaire

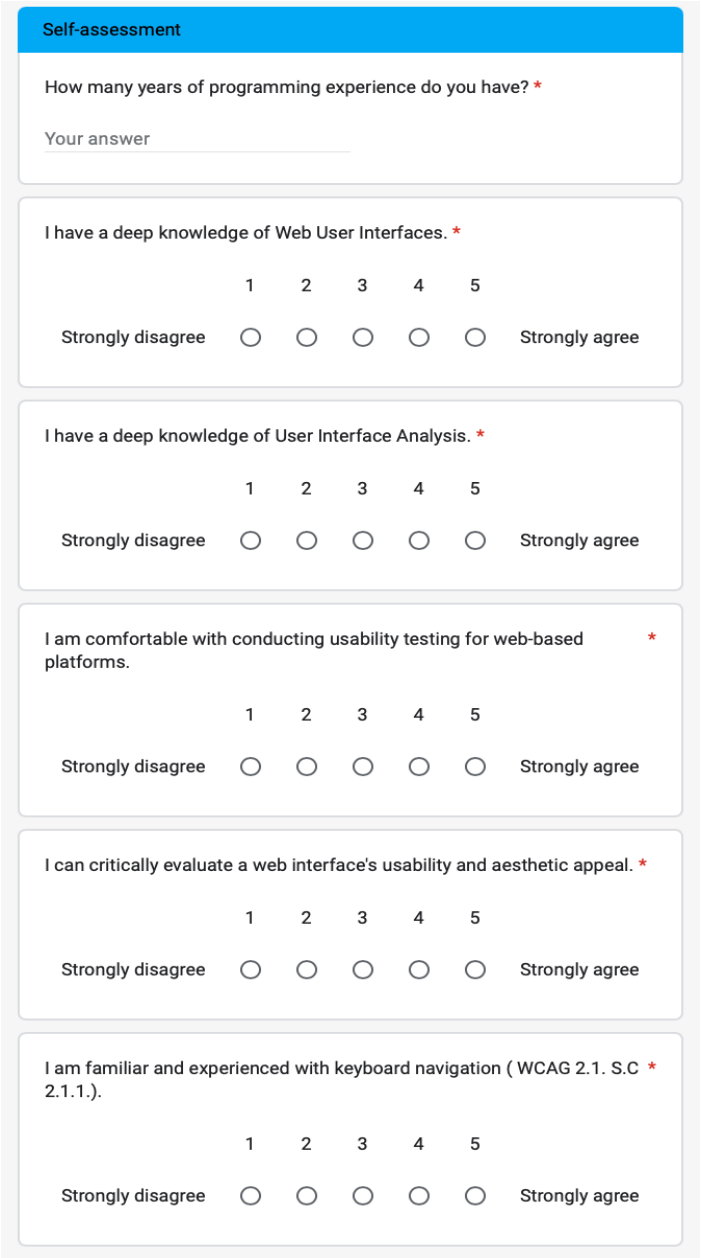
The following questionnaire was the initial questionnaire that test subjects need to fill before engaging in the usability evaluation of the proposed solution. All self-assessment statements are 5-level scale of agreement Likert items with 1 denoting strongly disagree and 5 denoting strongly agree. Google Forms is used to administer the survey.

Questions

Q1 What is your gender?
Q2 What is your age group?
Q3 What is the main language you use at home?
Q4 What is your field of study or professional background?

Self-assessment
S1 How many years of programming experience do you have?
S2 I have a deep knowledge of Web User Interfaces.
S3 I have a deep knowledge of User Interface Analysis.
S4 I am comfortable with conducting usability testing for web-based platforms.
S5 I can critically evaluate a web interface's usability and aesthetic appeal.
S6 I am familiar with and experienced with keyboard navigation (WCAG 2.1. S.C. 2.1.1).

Figure B.1 shows how the questionnaire is displayed for the self-assessment section.



Self-assessment

How many years of programming experience do you have? *

Your answer

I have a deep knowledge of Web User Interfaces. *

	1	2	3	4	5	
Strongly disagree	○	○	○	○	○	Strongly agree

I have a deep knowledge of User Interface Analysis. *

	1	2	3	4	5	
Strongly disagree	○	○	○	○	○	Strongly agree

I am comfortable with conducting usability testing for web-based platforms. *

	1	2	3	4	5	
Strongly disagree	○	○	○	○	○	Strongly agree

I can critically evaluate a web interface's usability and aesthetic appeal. *

	1	2	3	4	5	
Strongly disagree	○	○	○	○	○	Strongly agree

I am familiar and experienced with keyboard navigation (WCAG 2.1. S.C 2.1.1.). *

	1	2	3	4	5	
Strongly disagree	○	○	○	○	○	Strongly agree

Figure B.1: Self-assessment section of the initial questionnaire

B.2 Task List for Usability Evaluation

The following tasks were the tasks that test subjects needed to perform with the platform. Necessary materials required to successfully complete the tasks were provided to the test subject during the experiment. Task 4 is optional and intended for those who are familiar and confident with keyboard navigation.

Task 1 – Evaluating a Single Web User Interface Input

- Navigate to the homepage of the platform: https://wui-eval.web.app/.
- Provide a single web UI input of type URL. For illustrative purposes, you may use: https://vsr.informatik.tu-chemnitz.de/. Confirm your input to proceed.
- Select the following metrics for the input: **PNG file size** & **UIED Segmentation** and proceed to evaluate the input based on these selected metrics.
- Navigate to the access link for the evaluation results as soon as the link is ready.
- Once on the evaluation results page, locate the **UIED Segmentation** result. Click on the image output to open it in its original size in a new browser tab for detailed examination.
- Return to the evaluation results page and copy the URL of this page to the clipboard.
- Finally, start a new evaluation.

Task 2 – Evaluating Multiple Web User Interface (WUI) Inputs in Parallel

- Navigate to the homepage of the platform: https://wui-eval.web.app/.
- Please provide 2 WUI inputs, with one of type URL (e.g. https://vsr.informatik.tu-chemnitz.de/ and the other a PNG file.
- Select at minimum 2 metrics for each input. Proceed to evaluate the inputs.
- Navigate to the access link for the evaluation results of the PNG input as soon as the link is ready.
- Once the results for all selected metrics are ready, copy the URL of this evaluation results page to the clipboard.
- Finally, start a new evaluation.

Task 3 – Requesting for Metric Extension

- Navigate to the homepage of the platform: https://wui-eval.web.app/.
- Navigate to the Metric Extension Interface.
- Upload the necessary files (i.e. the provided materials) required for requesting metric extension.
- Provide the download link for the required computational model: https://github.com/delldu/ImageNima/blob/master/models/dense121_all.pt
- Provide an email address for further correspondence, e.g. max_zimmermann@gmail.com
- Confirm your metric extension request.

Task 4 (Optional) – Keyboard Navigation

Redo task 2, but this time solely with keyboard navigation.

B.3 Initial Questionnaire Responses

This section contains the responses of each test subject for the initial questionnaire described in B.1.

Q1	Q2	Q3	S1	S2	S3	S4	S5	S6
Male	25-34	Hindi	4	3	2	3	2	1
Male	25-34	English	6	2	2	1	3	1
Male	18-24	German	3	2	2	3	4	4
Male	45 or older	German	28	3	3	3	4	4
Male	25-34	German	3	3	3	4	4	1
Female	18-24	German	3	3	1	5	5	1
Female	18-24	Chinese	2	3	2	5	4	5
Male	25-34	Arabic	3	3	2	2	3	3
Male	35-44	English	10	4	4	4	4	4
Male	18-24	English	6	3	3	4	5	2
Male	25-34	Hindi	4	4	4	3	2	3
Male	25-34	German	6	4	3	4	2	4
Female	18-24	Korean	0.5	1	1	3	3	1

Table B.3.1: Questionnaire responses excluding Q4

Q1	Q2	Q4
Male	25-34	Computer Applications
Male	25-34	Computer Science
Male	18-24	Computer Science Student
Male	45 or older	Diplom Informatik
Male	25-34	Archaeology, Computer Science
Female	18-24	Computer Science
Female	18-24	Computer Science & Information Engineering
Male	25-34	Software Engineering
Male	35-44	Music Technology, Software Engineer
Male	18-24	Physics and Software Quality Assurance
Male	25-34	Master's student in web engineering, Frontend Developer
Male	25-34	Computer Science
Female	18-24	Literature and Social Science

Table B.3.2: Questionnaire responses for Q4

B.4 Task Completion Times

Gender	Age group	Task 1	Task 2	Task 3	Task 4
Female	18-24	2m 45s	1m 42s	1m 1s	not applicable
Male	18-24	4m 2s	3m 19s	2m 8s	4m 20s
Male	25-34	1m 46s	1m 58s	1m 34s	not applicable
Male	25-34	4m 23s	2m	1m 3s	not applicable
Male	25-34	1m 44s	1m 45s	1m 20s	not applicable
Male	35-44	2m 51s	2m 33s	1m 27s	2m 40s
Male	25-34	4m 33s	2m 36s	1m 16s	not applicable
Male	18-24	4m 12s	2m 24s	1m 15s	not applicable
Male	45 or above	4m 40s	3m 6s	4m 32s	1m 45s
Male	25-34	4m 20s	1m 32s	45s	not applicable
Male	25-34	3m 36s	5m	4m 45s	2m 57s
Female	18-24	3m 57s	1m 53s	1m 20s	2m 44s
Female	18-24	2m 43s	1m 20s	1m 30s	not applicable

Table B.4: Task completion times

B.5 SUS Evaluation Data

Table B.5 presents the individual responses to the SUS questionnaire.

Gender	Age range	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	SUS Score
Female	18-24	3	4	5	1	5	3	5	3	2	2	67.5
Male	18-24	1	1	5	3	4	1	4	1	5	3	75
Male	25-34	4	1	5	3	5	2	3	1	5	1	85
Male	25-34	4	1	5	2	4	3	4	2	4	4	72.5
Male	25-34	4	1	5	1	4	1	5	1	5	1	95
Male	35-44	4	2	4	1	5	1	5	1	4	2	87.5
Male	25-34	4	1	4	3	3	1	5	1	4	3	77.5
Male	18-24	2	3	4	2	5	1	3	2	3	2	67.5
Male	45 or above	2	4	2	4	4	2	3	3	2	4	40
Male	25-34	2	1	4	1	3	2	5	1	4	2	77.5
Male	25-34	3	2	3	2	3	2	4	2	4	1	70
Female	18-24	2	3	2	4	4	2	4	3	1	1	50
Female	18-24	3	2	4	4	5	1	5	1	4	2	77.5

Table B.5: SUS evaluation data

B.6 UEQ Evaluation Data

Table B.6.1 presents the individual responses of each test subject for the UEQ. The order of the items follows the original order of the questionnaire, which is publicly available to download at this link: https://www.ueq-online.org/Material/UEQ_All_Languages.zip.

Items																									
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
4	4	6	1	3	2	4	6	4	3	5	3	4	3	3	4	2	4	2	4	6	5	7	5	5	4
6	6	4	1	4	5	6	5	1	7	6	2	7	5	1	5	1	2	1	7	1	7	1	2	2	2
7	7	1	1	1	7	7	7	1	4	7	1	7	7	4	7	1	1	1	7	7	7	1	1	1	7
6	6	2	2	1	6	6	6	2	2	7	2	7	6	5	6	3	3	2	7	2	6	1	2	2	6
6	6	2	1	2	6	6	6	2	2	6	2	7	6	6	6	2	2	1	7	1	6	1	2	2	6
5	6	4	2	2	4	6	6	1	4	7	1	6	6	6	6	1	2	1	7	2	7	2	4	3	5
4	6	4	1	2	4	5	6	2	3	6	2	6	4	4	5	3	3	3	5	4	5	2	3	2	6
5	4	6	2	4	2	5	7	1	4	5	3	5	4	4	5	3	3	4	6	3	6	4	5	3	3
4	5	5	1	2	4	5	7	1	4	4	2	6	5	6	5	4	4	1	7	3	7	2	7	5	6
4	3	6	5	6	3	3	6	2	6	6	2	3	5	5	4	4	3	2	5	5	5	5	3	5	6
5	6	5	2	2	3	6	7	2	5	4	2	6	5	3	6	2	4	1	3	2	7	5	5	4	2
4	3	6	4	6	5	7	3	1	1	6	2	3	4	6	5	4	3	2	7	1	7	2	2	4	6
4	4	4	2	2	4	5	5	1	5	6	1	6	6	2	5	6	4	2	7	1	7	2	4	1	4

Table B.6.1: UEQ raw evaluation data

Table B.6.2 shows the results for each item of the UEQ.

Item	Mean	Variance	Std. Dev.	Left	Right	Scale
1	0,9	1,1	1,0	annoying	enjoyable	Attractiveness
2	1,1	1,7	1,3	not under-standable	understandable	Perspicuity
3	-0,2	2,9	1,7	creative	dull	Novelty
4	2,1	1,6	1,3	easy to learn	difficult to learn	Perspicuity
5	1,2	2,8	1,7	valuable	inferior	Stimulation
6	0,2	2,4	1,5	boring	exciting	Stimulation
7	1,5	1,3	1,1	not interesting	interesting	Stimulation
8	1,9	1,2	1,1	unpredictable	predictable	Dependability
9	2,4	0,8	0,9	fast	slow	Efficiency
10	0,2	2,8	1,7	inventive	conventional	Novelty
11	1,8	1,0	1,0	obstructive	supportive	Dependability
12	2,1	0,4	0,6	good	bad	Attractiveness
13	1,6	2,1	1,4	complicated	easy	Perspicuity
14	1,1	1,2	1,1	unlikable	pleasing	Attractiveness
15	0,2	2,7	1,6	usual	leading edge	Novelty
16	1,3	0,7	0,9	unpleasant	pleasant	Attractiveness
17	1,2	2,2	1,5	secure	not secure	Dependability
18	1,1	0,9	1,0	motivating	demotivating	Stimulation
19	2,2	0,9	0,9	meets expecta-tions	does not meet expectations	Dependability
20	2,1	1,9	1,4	inefficient	efficient	Efficiency
21	1,1	4,1	2,0	clear	confusing	Perspicuity
22	2,3	0,7	0,9	impractical	practical	Efficiency
23	1,3	3,7	1,9	organized	cluttered	Efficiency
24	0,5	2,9	1,7	attractive	unattractive	Attractiveness
25	1,0	2,2	1,5	friendly	unfriendly	Attractiveness
26	0,8	2,8	1,7	conservative	innovative	Novelty

Table B.6.2: UEQ results for each item of the questionnaire

B.7 AttrakDiff Evaluation Data

Table B.7 presents the individual responses from each test subject for the AttrakDiff questionnaire. The order of the items follows the order used in the example survey available at: https://esurvey.uid.com/survey/#uid-test-attrakdiff-03.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
7	4	4	5	5	5	4	6	6	4	3	5	5	5	6	5	3	4	5	5	5	3	4	5	4	4	3	6
5	6	6	5	6	7	6	7	7	6	6	6	4	5	6	6	6	6	7	6	6	6	6	6	6	6	5	6
6	6	6	6	6	6	6	7	7	7	7	7	7	6	7	7	7	7	7	7	7	6	7	7	4	7	7	7
2	5	3	3	3	4	3	6	4	5	4	5	4	5	4	3	3	3	4	2	4	4	4	3	4	3	5	4
1	4	5	6	6	6	1	7	5	7	3	6	7	4	4	6	4	5	6	6	4	4	5	4	1	4	6	7
2	4	6	3	7	6	4	7	4	6	3	6	5	4	4	6	5	3	7	5	4	3	2	4	1	5	2	6
2	4	6	6	5	6	4	6	5	6	4	6	4	5	4	5	5	6	6	6	6	5	6	5	3	6	6	7
1	7	3	7	3	6	2	6	3	6	1	2	2	1	1	3	1	4	4	5	5	6	6	1	6	4	4	5
4	3	4	6	6	6	5	5	5	6	4	5	4	4	1	5	4	6	5	5	4	4	5	4	3	4	5	5
4	5	3	1	5	4	5	5	4	4	2	5	4	4	4	4	4	4	7	5	4	4	1	5	3	4	2	6
2	3	4	5	5	6	3	6	6	6	4	6	4	5	2	6	5	2	6	5	4	1	3	2	2	4	2	7
6	6	6	6	6	6	6	6	6	7	4	6	4	4	4	4	4	6	6	7	6	4	6	6	4	6	4	4
4	4	2	4	4	4	2	6	2	3	2	6	4	4	4	6	3	4	6	3	4	4	4	2	4	3	4	5

Table B.7: AttrakDiff raw evaluation data

B.8 Computation Times Statistics of Feature Congestion and Subband Entropy Metrics

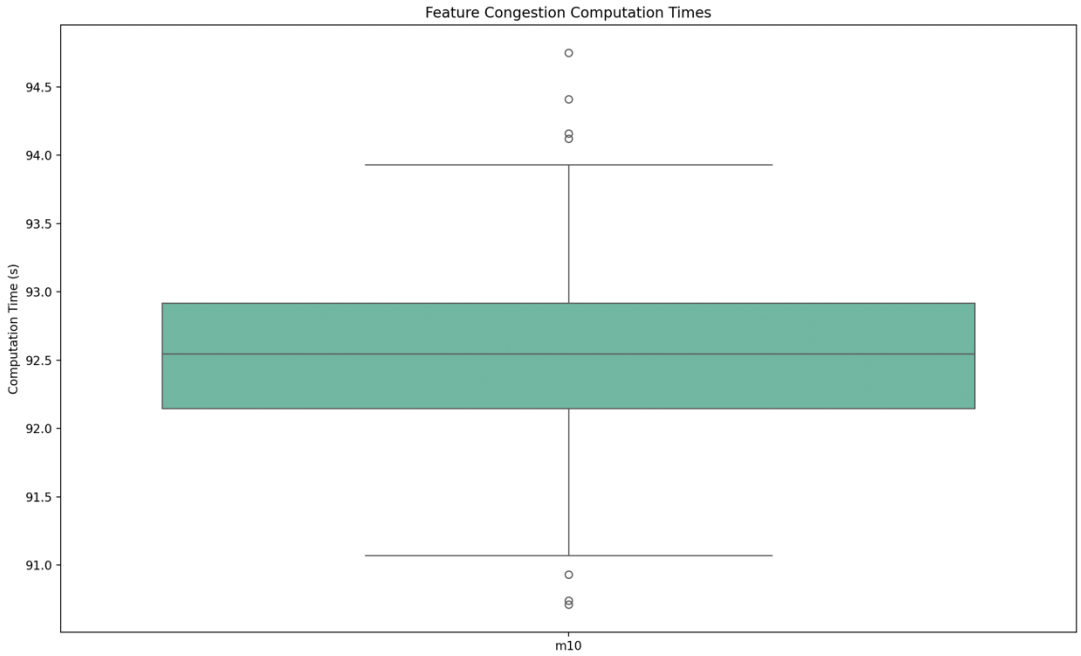


Figure B.8.1: Computation times of the feature congestion metric

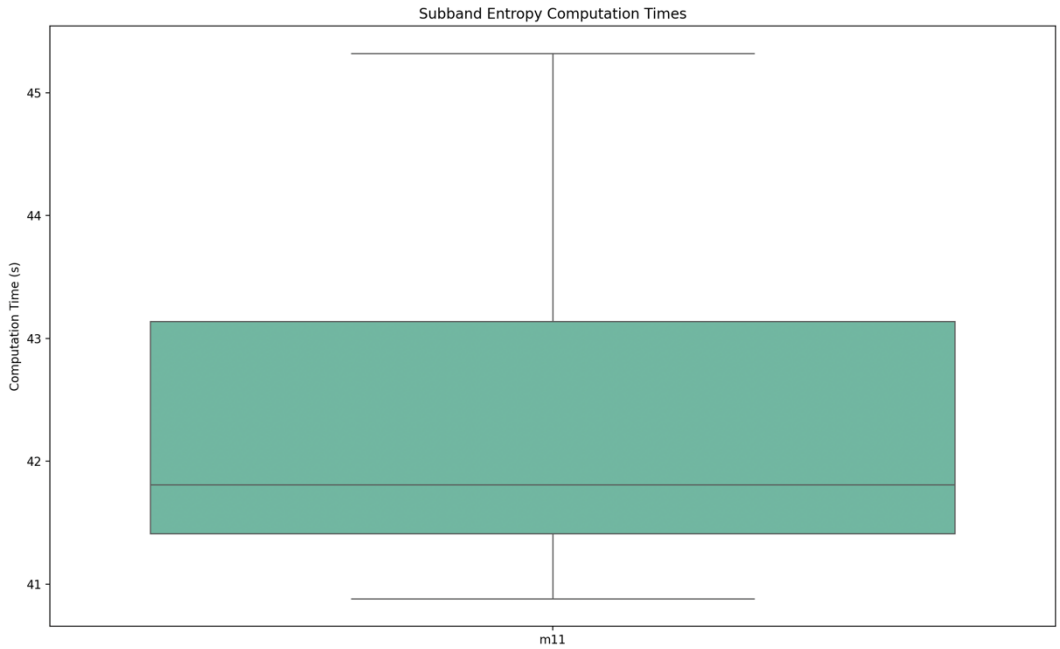


Figure B.8.2: Computation times of the subband entropy metric

TECHNISCHE UNIVERSITÄT CHEMNITZ

Zentrales Prüfungsamt
Selbstständigkeitserklärung

Name:	Liusnando	**Bitte beachten:** 1. Bitte binden Sie dieses Blatt am Ende Ihrer Arbeit ein.
Vorname:	Calvin	
geb. am:	15.01.2000	
Matr.-Nr.:	708955	

Selbstständigkeitserklärung*

Ich erkläre gegenüber der Technischen Universität Chemnitz, dass ich die vorliegende Masterarbeit selbstständig und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel angefertigt habe.

Die vorliegende Arbeit ist frei von Plagiaten. Alle Ausführungen, die wörtlich oder inhaltlich aus anderen Schriften entnommen sind, habe ich als solche kenntlich gemacht.

Diese Arbeit wurde in gleicher oder ähnlicher Form noch nicht als Prüfungsleistung eingereicht und ist auch noch nicht veröffentlicht.

Datum: 15.04.2024 Unterschrift: ……

---

* Statement of Authorship

I hereby certify to the Technische Universität Chemnitz that this thesis is all my own work and uses no external material other than that acknowledged in the text.

This work contains no plagiarism and all sentences or passages directly quoted from other people's work or including content derived from such work have been specifically credited to the authors and sources.

This paper has neither been submitted in the same or a similar form to any other examiner nor for the award of any other degree, nor has it previously been published.